23 July 2025

InFuse: In-Situ Material Injection for 3D Printing

Hao Xu[1], Shuai Feng[1], Yi Xiong[2], Liang He[3], Wenpeng Xu[4], Haisen Zhao[1]

1. Shandong University
2. Southern University of Science and Technology
3. University of Texas at Dallas
4. Henan Polytechnic University

Abstract

We present InFuse, a hybrid manufacturing technique that synergizes in-situ material injection with 3D Printing to enable the rapid fabrication of isotropic, complex geometries and multi-material composites. Unlike the conventional "printing-then-injection" paradigm, we introduce a novel volume decomposition algorithm based on a directed acyclic graph (DAG) that optimizes manufacturing sequences by jointly considering 3D Printing and injection constraints. This algorithm is featured in an interactive design tool we developed for the user to "control" custom models for injection and 3D printing. Additionally, we developed a modified 3D printer that dynamically interleaves 3D Printing and injection processes, automating the fabrication of isolated cavities and non-connected channels, and eliminating manual intervention. To validate the abroad applicability of InFuse, we conduct tests on InFuse using geometries with varying levels of geometric complexity from a wide range of applications across multiple fields, including soft-bodied robotics, artistic creation, mold manufacturing and so on. We then perform careful physical validations to assess the effectiveness and reliability of the results obtained from the series of experiments.

Keywords

3d printing, additive manufacturing, components, circuits, devices and systems, computing and processing, engineering profession, general topics for engineers, human-computer interaction

Posted on 23 July 2025 — CC-BY 4.0 — This is a preprint and has not been peer reviewed. Data may be preliminary. — https://doi.org/10.36227/techrxiv.175329147.74597084/v1

InFuse: In-Situ Material Injection for 3D Printing

Hao Xu
Shandong University
Qingdao, China
haoxu5640@gmail.com

Shuai Feng
Shandong University
Qingdao, China
sfeng0253@gmail.com

Yi Xiong
Southern University of Science and Technology
Shenzhen, China
xiongy3@sustech.edu.cn

Liang He
University of Texas at Dallas
Richardson, United States
liang.he@utdallas.edu

Wenpeng Xu
Henan Polytechnic University
Jiaozuo, China
wpxu08@gmail.com

Haisen Zhao*
Shandong University
Qingdao, China
haisenzhao@sdu.edu.cn

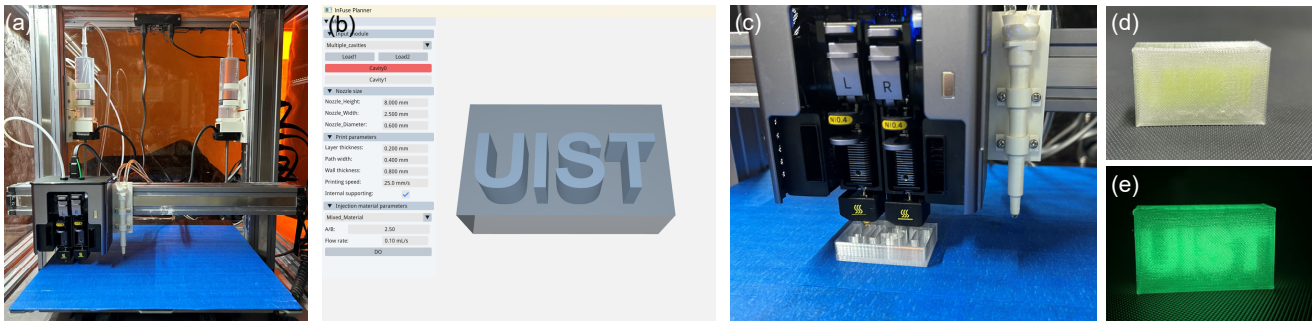


Figure 1: InFuse's printing pipeline overview. (a) A modified FDM 3D printer equipped with an additional materials injection module. (b) The InFuse slicer interface, where the injection cavities in the input model are non-interconnected. (c) The hybrid manufacturing process in progress. (d) A final manufactured model with a seamless surface, free of visible sprue openings. (e) The injected material exhibits a fluorescent effect.

ABSTRACT

We present InFuse, a hybrid manufacturing technique that synergizes in-situ material injection with 3D Printing to enable the rapid fabrication of isotropic, complex geometries and multi-material composites. Unlike the conventional "printing-then-injection" paradigm, we introduce a novel volume decomposition algorithm based on a directed acyclic graph (DAG) that optimizes manufacturing sequences by jointly considering 3D Printing and injection constraints. This algorithm is featured in an interactive design tool we developed for the user to "control" custom models for injection and 3D printing. Additionally, we developed a modified 3D printer that dynamically interleaves 3D Printing and injection processes, automating the fabrication of isolated cavities and non-connected channels, and eliminating manual intervention. To validate the abroad applicability of InFuse, we conduct tests on InFuse using geometries with varying levels of geometric complexity from a wide range of applications across multiple fields, including soft-bodied robotics, artistic creation, mold manufacturing and so on. We then perform careful physical validations to assess the effectiveness and reliability of the results obtained from the series of experiments.

CCS CONCEPTS

• **Computing methodologies** → **Shape modeling**; *Graphics systems and interfaces.*

KEYWORDS

Additive manufacturing, Material injection, Sequence planning, 3D printing

ACM Reference Format:
Hao Xu, Shuai Feng, Yi Xiong, Liang He, Wenpeng Xu, and Haisen Zhao. 2025. InFuse: In-Situ Material Injection for 3D Printing. In . ACM, New York, NY, USA, 20 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

*corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

3D Printing (3DP), also known as additive manufacturing (AM), has revolutionized fabrication by enabling unprecedented geometric complexity and customization, empowering industries to prototype and produce intricate designs that were previously unachievable

with traditional methods [25]. However, 3DP still faces critical limitations that hinder its broader adoption [1]. First, the anisotropic nature of layer-by-layer deposition often results in poor mechanical performance, particularly in polymer-based 3DP (e.g. FDM, SLA). Second, the point-wise material deposition in traditional 3DP is inherently slow, especially for large-scale objects or mass-production. Third, the stair-stepping effect and layer adhesion issues degrade surface quality, necessitating post-processing. These shortcomings collectively constrain the full exploitation of 3DP's potential across broader industrial and creative domains.

To tackle these limitations, a wide array of incremental improvement approaches, optimization-based technological advancements and post-processing strategies have become imperative. For example, methods like directed energy deposition (DED) with controlled grain structuring [45], fiber-reinforcement [46], and curved slicing [21, 53], in-situ and post-build heat treatments in metal 3DP processes [48] can diminish directional mechanical weaknesses due to anisotropic material distribution. Parallel processing with multi-head printing [34] and area-wise curing techniques such as LCD/DLP for Vat photopolymerization [7] have been suggested to accelerate 3DP processes and enhance productivity. Although these methods have shown remarkable progress, their effectiveness to further significant improvement remains constrained by the the gradual incremental forming paradigm of 3DP. To overcome these intrinsic limitations of 3DP, hybrid approaches that incorporate other manufacturing strategies have been employed to enhance 3DP processes. For example, utilizing CNC machining with 3DP, known as Additive and Subtractive Hybrid Manufacturing (ASHM), has been proposed to significantly improve the surface quality on the order of magnitude scale by alternating between additive and subtractive processes [18]. Similar to ASHM, the hybrid manufacturing approach holds great promise for addressing diverse challenges in 3DP, with this paper placing particular emphasis on injection molding. Integrating injection molding into the 3D printing process can significantly alleviate the aforementioned limitations, with a particularly pronounced impact on addressing anisotropy and enhancing fabrication efficiency.

Injection molding produces parts by injecting material into a mold cavity, which enables cost-effective mass production with short cycle times, allowing the use of diverse materials with isotropic material distribution. To integrate these injection molding advantages into 3DP, the basic idea is to substitute the internal volume of printed objects with injection materials via a significantly faster process, as indicated by about a 50% efficiency boost relative to pure 3DP in [42]. Studies have investigated enhancing 3DP parts through the sequential "**printing-then-injection**" approach [8], like eggshell moulding[1] and the continuous fiber injection process (CFIP) [16]. Here, a structural model is first created via 3DP, followed by injecting epoxy resin or other materials to enhance the strength and rigidity of the printed parts. However, this "printing-then-injection " approach greatly limits geometric adaptability, requiring *fully interconnected material flow channels*, and *manually seal the pre-reserved vent holes or introduce additional flow channels to address air entrapment issues* (refer to Figure 2). For arbitrary

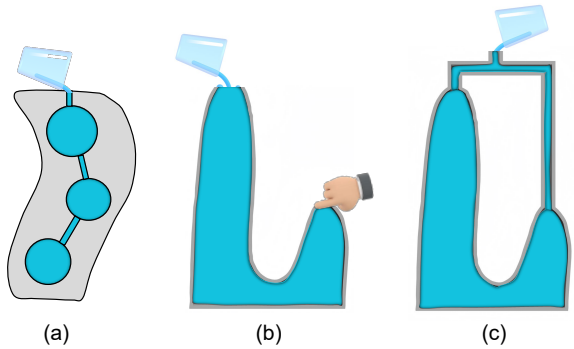


Figure 2: Hybrid manufacturing of 3D printing and material injection. (a) For the material flow channel, cavities need interconnection and should be free of air gaps; otherwise, the injected material won't fill them completely. (b) Exterior air holes must be reserved and manually sealed. (c) Alternatively, additional flow channels are required.

models, the internal injection channels must be carefully designed, as demonstrated by Yan et al. [54], who use Triply Periodic Minimal Surface (TPMS) to design internal structures for injection channels. To fully generalize to arbitrary models, a straightforward approach is to investigate in-situ material injection in 3DP, interleaving the process of 3DP and material injection, where specific internal injection channels do not require pre-designing.

Some preliminary attempts have been made to showcase the potential of in-situ material injection in 3DP. For example, the Ulticast method suggests an add-on for 3D printers to fabricate soft robotic parts[2], while Fraunhofer IPA proposes 3D printing freeform casting which combines the benefits of injection molding and 3D printing[3]. These efforts merely illustrate fabrication methods for particular cases like basic silicone pieces or 2.5D elements. Moreover, no definitive proof suggests they have developed a comprehensive printing and injection planning algorithm for general models.

we propose ***InFuse***, the first fully automated hybrid manufacturing system that seamlessly integrates in-situ material injection with 3DP through a computational pipeline and accompany hardware solution. This pipeline eliminates manual intervention by synchronizing material deposition (3DP) and functional infilling (injection) within a unified workflow. Users can input any pre-designed 3D model and specify the parts of the model to be filled with injection materials. Our computational pipeline then automatically generates an efficient manufacturing sequence for 3D printing and material injection, which is subsequently executed on the in-situ injection-printing platform. Our approach contributes:

- The automation of hybrid 3DP-injection for complex geometries without dedicated channels, using a DAG-based algorithm to co-optimize sequences of 3D printing and material injection. Moreover, a postprocessing step generates a feasible print path to mitigate interlayer gaps that may

[1]https://www.tctmagazine.com/additive-manufacturing-3d-printing-news/eggshell-moulding-3d-printing-hidden-application/

[2]https://3dprint.com/154931/tu-delft-3d-printed-soft-robotics

[3]https://all3dp.com/fraunhofer-ipa-combine-3d-printing-injection-molding-advantages/?utm_source=push

cause fluid leakage. The above computational algorithms are integrated into InFuse planner for the control of injection and 3D printing models.
- An modified 3D printer that automatically switches between printing and injection modes, creating internal channels and locking air vents in-situ enabling post-process-free fabrication of complex multi-height geometries.
- A suite of applications, including soft robotics, artistic molds, and functional composites, to demonstrate the potential for generating multi-material functional structures, offering a versatile material selection for filling and allowing staged material injection.

2 RELATED WORK

2.1 Hybrid manufacturing

Many hybrid manufacturing techniques have been proposed to address the issues in traditional 3DP. One class of work combines subtractive manufacturing with additive manufacturing, referred to as additive and subtractive hybrid manufacturing (ASHM) [17, 19, 22, 24, 41]. By first performing 3DP, followed by subtractive manufacturing (SM, CNC machining) on the surface of printed parts, the surface quality of the 3D shape is improved. Similar to our problem, the AM-then-SM sequence planning problem exists in ASHM, where manufacturing constraints such as nozzle/cutter accessibility and self-supporting must be considered. Numerous studies have focused on minimizing the number of AM-then-SM sequences required to manufacture 3D shapes [10, 27, 38, 58]. Another class of work, referred to as low-fidelity fabrication techniques, uses rapid manufacturing methods such as laser cutting [9, 43] or LEGO blocks [13, 39] to quickly create internal structures without accuracy requirements, followed by 3DP to produce the external structure. To improve efficiency and reduce seams, this type of work often investigates the optimal way to segment 3D shapes. For example, CofiFab [43] established an optimization model to calculate the maximum machinable convex polyhedron of the target shape. We adopt a novel hybrid approach that integrates 3DP and injection to address the problems of low efficient and anisotropy in 3D printing.

2.2 Molding aided 3DP

Printing-then-Injection. This type of work performs injection in a single step after 3DP. Typical examples include using triply periodic minimal surfaces (TPMS) to define internal channel structures [54], which enables self-supporting printing while enhancing strength by injecting external materials; generating voxel matrix channels to enable objects to have magnetophoretic surfaces after injection. [55]; pre-processing of internal hollow structures to maximize part strength following injection [8]. Although the printing-then-injection strategy has been validated for superior mechanical performance through material mechanics analysis [32], the above work pre-define the injection channel before 3DP, overlooking the issue of air entrapment encountered during injection, and requiring manual operation, which limits its practical application.

In contrast, InFuse fully considers the manufacturing constraint caused by air entrapment and is capable of automatically generating manufacturable printing-then-injection sequences for arbitrary shapes, without the need for manual operation.

Interleaving Printing and Injection. Another type of work completes the manufacturing through multiple alternating printing-then-injection sequences. They are similar to in-situ additive manufacturing, which refers to 3DP based on existing components, has been widely studied in numerous fields such as architecture [23], industry [11], healthcare [14], and space [51]. In everyday scenarios, the human-computer interaction field has also proposed many novel ideas for in-situ additive manufacturing [12, 33, 44, 46]. Notably, a 3D printing software called Scrappy has been proposed for multi-stagein-situ material injection, in which internal infill structures are replaced with scrap material to reduce material and energy consumption [50]. For 3DP combine injection, some works adopt similar approaches. For instance, Z-pinning across layers within the part to enhance tensile strength along the Z-axis [20]; adding injection attachments to 3D printers enables in-situ fabrication of soft robotic parts [49] and free-form casting [6].

However, these methods are applied to specific cases and lack of sequence planning algorithm with considering air entrapment constraint. While Infuse breaks this limitation by generating manufacturable printing-then-injection sequences for arbitrary shapes.

2.3 3DP aided molding

Another category of research leverages the capability of 3DP to fabricate complex geometries for mold creation, followed by injection to produce the final parts. The 3DP structures are removed—either by disassembly or dissolution—leaving only the internal injection material. This type of work often focuses on the design of the mold itself [2–5], such as optimizing mold structures based on height fields under geometric and topological constraints to achieve reusable molds [3] and flexible molds [4], while ensuring detachability. Alternatively, some end-to-end process planning frameworks have been proposed in recent years [29, 30], incorporating direction optimization, injection parameter selection, etc.

Although these works do not retain the 3DP materials like InFuse, they demonstrate that shapes produced via injection exhibit excellent isotropy and manufacturing efficiency. The use of materials such as low-melting-point alloys [30] and ceramics [52] for injection also shows great potential for widespread applications.

2.4 Beyond-layered 3D printing

Traditional 3D printing adopts a 2.5D strategy to fabricate shapes. In recent years, some works have leveraged graph-based methods to group layers based on local features (called beyond layers [35]). Minimizing total extrusionless travel distance is a common objective [31, 36, 37, 56], where sliced layers are treated as nodes in a dependency graph, turning the problem into a Traveling Salesman Problem (TSP) solvable via heuristics [31, 36, 37] or Monte Carlo Tree Search [56]. ACAP [57] instead aims for optimal path continuity by encoding accessibility and inter-layer dependency constraints into the graph, formulating the ordering as a Precedence-Constrained Minimum Path Cover Problem (PC-MPC). In the domain of human-computer interaction, the recent TRAvel Slicer [26] extends ACAP to solid shapes using Fermat spiral infill, enabling more efficient printing.

Similar to ACAP, we produce shells in the 3DP phase and use a graph-based approach. Unlike ACAP, which focuses solely on

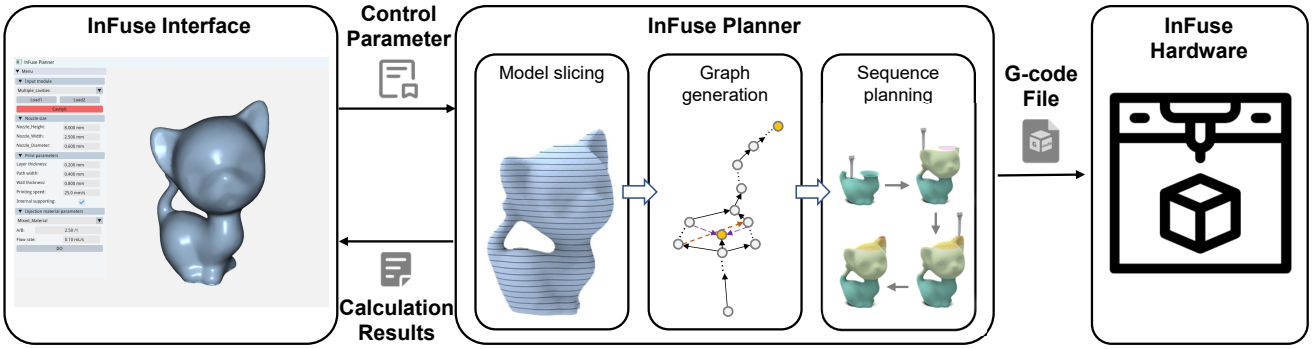


Figure 3: An overview of the InFuse System.

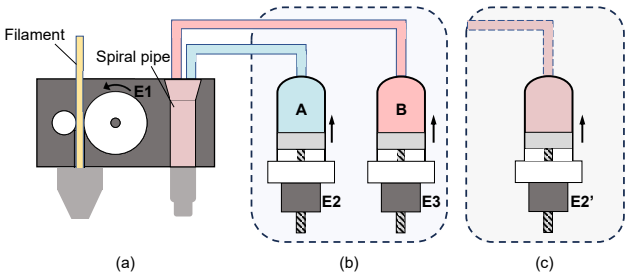


Figure 4: Schematic of the platform design. (a) The printhead that integrates the fused deposition and injection modules. (b) The dual-component extrusion module for injection. (c) The single-component extrusion module for injection.



Figure 5: Our machine was modified from a desktop 3D printer (Snapmaker 2.0 Modular 3-in-1 3D Printer). The overall device (left). The injection module is connected with the printhead in parallel (upper right). The lower right corner demonstrates the printing process and injection process.

constraints in 3DP, we also address air entrapment issues during injection. This extends our method to hybrid 3DP and injection technology with a novel subgraph merging strategy.

3 INFUSE OVERVIEW

We present InFuse, a fully automated hybrid manufacturing system that seamlessly integrates in-situ material injection with 3D printing (3DP) through co-designed computational and hardware innovations. By synchronizing material deposition (3DP) and functional filling (injection) within a unified workflow.

The InFuse system consists of there main components: a hardware platform, a manufacturing sequence planner and a user interface. On the hardware side, we design and implement an **in-situ injection-printing platform** (section 4), which fulfills the requirements of hybrid printing-injection manufacturing by supporting precise, synchronized execution of both processes.

In addition, we develop a software tool called **injection-printing sequence planner** (see Figure 3). Given a user-designed model as input, the planner outputs machine-executable G-code compatible with the hardware platform. Internally, the planner incorporates a sequence planning algorithm that determines the optimal order of printing and injection steps, ensuring manufacturability while improving process efficiency (section 5 and section 6). To enhance user control, we also provide an **user interactive interface** (section 7) that allows users to adjust key parameters and fine-tune the resulting fabrication behavior.

4 IN-SITU INJECTION-PRINTING PLATFORM

This section addresses the hardware architecture of InFuse. Initially, the platform framework is designed for the in-situ injection-printing platform (subsection 4.1). Next, the adaptation of a desktop 3D printer is described (subsection 4.2). Lastly, the fabrication workflow for integration of 3DP with in-situ material injection is introduced (subsection 4.3).

4.1 Platform design

InFuse uses a co-located dual-nozzle module integrated into a shared print head assembly. Both extrusion mechanisms are rigidly connected to a unified motion platform, yet have independent material delivery paths (Figure 4(a)), where the first nozzle uses a standard Fused Deposition Modeling (FDM) extruder for thermoplastic filaments (e.g., PLA, ABS). It has a nozzle with precise volumetric flow control through a geared stepper motor (E1 in Figure 4(a)).

The second nozzle features a new material injection module that can serve as a dual-component extrusion module for reactive materials like A/B epoxy resin. It uses two lead-screw stepper motors (E2 and E3 in Figure 4(b)) to synchronize syringe motions for mixing in a static nozzle, ensuring uniform deposition. This setup allows precise control of material ratios and integrates with the FDM process. Moreover, it can be adapted to single-component materials (e.g., silicone, UV resin) by using just one lead-screw mechanism (E2' in Figure 4(c)), enhancing versatility for various applications.

4.2 Platform implementation

Since integrated 3D printing and material injection equipment is rare on the market, we chose to modify a standard three-axis 3D printer. Although this approach only allows processing of plastic materials such as PLA, it does not affect the generality of InFuse. Specifically, we modified a desktop 3D printer (*Snapmaker 2.0 Modular 3-in-1 3D Printer*) by adding an external injection mixing nozzle parallel to the 3D printing nozzle. This printer is capable of switching between subtractive manufacturing and additive manufacturing by replacing different modules. Additionally, it supports a dual-extrusion module for 3D printing.

The key challenge in this modification was that mixing A/B liquid components required two additional stepper motors. Interestingly, when the printer operates in subtractive mode, it features an extra clamp rotation motor, and in printing mode, it supports dual-extrusion printing. Leveraging this, we repurposed the built-in stepper motors—specifically, the clamp rotation motor from subtractive mode and one of the dual-extrusion motors from printing mode—by replacing them with a through-screw motor to drive the A/B liquid dispensing mechanism. This approach effectively "tricks" the system, allowing us to achieve our goal simply by controlling the original motor commands. The modified device is shown in Figure 5, where the overall device is shown on the left, a zoomed-in view of the printhead is shown on the top right, and the printing and injection processes are shown on the bottom right, respectively.

4.3 Fabrication workflow

The fabrication workflow begins with 3DP to build the outer shell of the model. Upon reaching the first designated injection point, liquid material is injected through a nozzle until the cavity is fully filled up to the injection interface. This point is then sealed using 3DP, and the process continues with alternating 3DP and injection steps until all designated cavities have been filled. Finally, the last injection point is sealed with a layer of 3DP, completing the fabrication process. This procedure is illustrated in Figure 6.

In mathematical terms, InFuse fabricates a 3D object M through a sequence of interleaved 3DP and injection operations, formally represented as: $\{P_1, I_1, P_2, I_2, \ldots, P_n, I_n, P_{n+1}\}$, where P_i and I_i denote the 3D printing and material injection steps, respectively. P_{i+1} represents the final 3DP sealing layer. The total number of alternations n depends on the spatial distribution of cavities in the model's geometry.

5 INFUSE SEQUENCE PLANNER

This section presents a sequence planning algorithm tailored to the InFuse manufacturing constraints. We begin by formally defining the sequence planning problem and the associated manufacturing constraints (subsection 5.1). To address this problem, we formulates both geometric and process dependencies using a directed acyclic graph (DAG). Given an input model M , we first convert its sliced contours into a DAG to capture the necessary dependencies and guide the manufacturing sequence (subsection 5.2).

Due to the potentially large scale of the graph, we introduce a graph simplification method to reduce computational complexity while preserving critical structure information (subsection 5.3). Finally, we apply a beam search algorithm, guided by topological sorting, to identify an optimal manufacturing sequence that satisfies all constraints (subsection 5.4).

5.1 Problem statement

Model M is represented by a triangle mesh. We aim to generate a feasible injection-printing sequence that minimizes the overall fabrication time. However, since the number of alternations between 3DP and material injection is inherently determined by the model's geometric structure (see details in subsection 5.2). As a result, our optimization primarily targets reducing the time consumed by the 3DP phase.

We adopt the beyond layer fabrication strategy [35] to optimize 3DP process. Initially, the input model M is first sliced to produce a set of contours $\mathbf{c} = \{c_1, c_2, \ldots, c_m\}$, where m represents the total number of contours. The key objective is to determine an optimal 3DP sequence σ by strategically reordering the contours in $\mathbf{c}$ to minimize the overall fabrication time $T(\sigma)$. This optimization problem can be formally defined as follows:

$$\min_{\sigma \in S_m} T(\sigma) = \frac{\sum_{i=1}^{m-1} \|\text{pos}(c_{\sigma(i)}) - \text{pos}(c_{\sigma(i+1)})\|}{v_{travel}},$$
$$\text{s.t.} \begin{cases} u_i \in \{1, 2, \ldots, m\}, & \forall i = 1, 2, \ldots, m \\ u_i \neq u_j, & \forall i \neq j \\ u_i < u_j, & \forall (i, j) \in D \end{cases} \tag{1}$$

where S_m represents the set containing all contour permutations and $u_i \in \mathbb{R}^+$ denotes the sequential position of contour c_i in the printing sequence, with the relation $u_i = \sigma^{-1}(i)$, meaning that if $\sigma(k) = i$, then $u_i = k$. $\text{pos}(\cdot)$ represents the geometric position of a contour and v_{travel} corresponds to the nozzle's idle travel speed. $D = \{(i, j) \mid c_i \rightarrow c_j\}$ is the set of dependency pairs, ensuring that if $(i, j) \in D$, contour c_i must precede contour c_j in the sequence. Sequence planning must satisfy several constraints to maintain the correctness and manufacturability of the printing process, as detailed below:

Unique visit constraint. To ensure the completeness of M, every contour must be printed exactly once. This requires that each contour c_i is assigned a unique sequential position u_i in the printing order, ensuring that all contours are included and no contour is revisited. Specifically, u_i must belong to the set $\{1, 2, \ldots, m\}$ for all $i = 1, 2, \ldots, m$, and for any two distinct contours c_i and c_j, their positions must be different, i.e., $u_i \neq u_j$ for all $i \neq j$. These constraints ensure that each contour is traversed exactly once, maintaining the integrity of the manufacturing process.

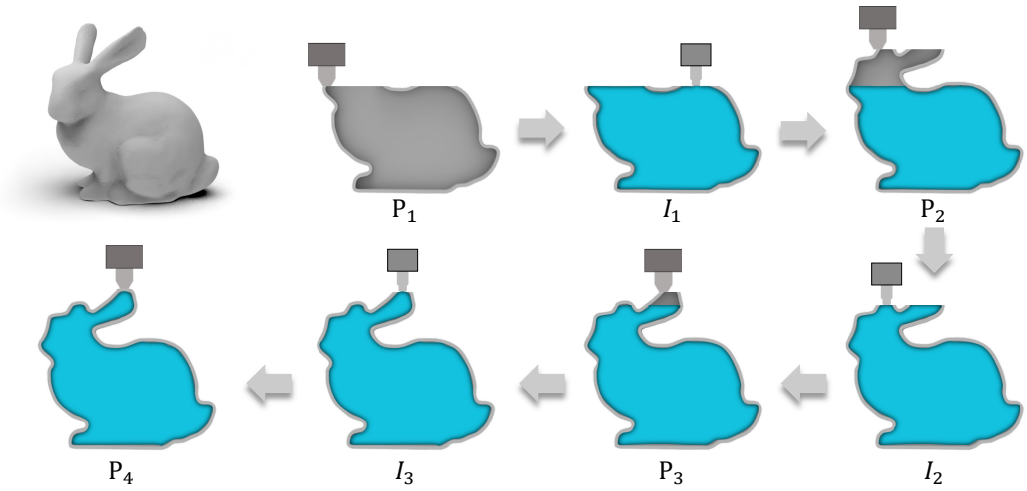


Figure 6: The hybrid manufacturing process of 3D printing and injection. The process begins with 3DP printing the outer shell of the model up to the first injection port. An injection nozzle is then employed to fill the internal cavity. After injection, 3DP is used to seal the port, and printing continues. This cycle repeats until the final injection port is sealed by 3DP at the end of the process.

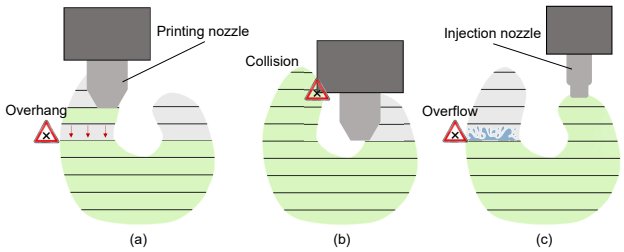


Figure 7: Illustration of manufacturing constraints under three failure scenarios. (a) Height dependency constraint: 3D printing must satisfy the height condition to prevent overhanging of printed contours. (b) Collision-free constraint: A collision occurs between the printhead and previously printed regions. (c) Injection dependency constraint: During injection, liquid may leak from locations lower than the injection port due to gravity-driven flow.

Height dependency constraint. For the 3D printing process, the height dependency constraint ensures that contours in different layers with a vertical alignment are printed in the correct sequence. Specifically, if contour c_i is directly below contour c_j, with c_i providing support for c_j, then c_i must be printed before c_j to prevent potential collapse or instability in the structure, as shown in Figure 7(a). This dependency reflects the requirement that the lower contour must be fully printed before its upper counterpart to ensure proper support during the manufacturing process. Thus, whenever a contour c_i is positioned directly below another contour c_j, a dependency constraint $c_i \rightarrow c_j$ is imposed, enforcing that c_i is printed before c_j.

Collision-free constraint. Since we adopt a beyond layer approach for manufacturing, it is crucial to ensure that the nozzle does not collide with other contours during the printing process. Specifically, let $\mathcal{N}$ denote the nozzle. If a collision occurs between the nozzle and contour c_j while printing contour c_i (see Figure 7(b)), i.e., $\mathcal{N}(c_i) \cap c_j \neq \emptyset$, a dependency constraint $c_i \rightarrow c_j$ is introduced to enforce that c_j must be printed after c_i, thereby preventing collisions.

Injection dependency constraint. The injection dependency constraint applies when contour c_i is designated as the injection port. In this case, it is essential that all other contours, c_j, are printed at least up to the same height as c_i. If exist contour c_j is printed lower than c_i, the injection material, which flows horizontally, may overflow from the lower regions of c_j, thereby resulting in a manufacturing failure, as shown in Figure 7(c). Therefore, to ensure the proper injection process, a dependency constraint $c_j \rightarrow c_i$ is imposed.

5.2 Dependency graph initialization

The input to this step is a single-cavity model M (see Figure 3 for the input model) or a multi-cavity model $M^{\dagger}$ and $M^{\ddagger}$ (see Figure 1 of appendix text), where $M^{\dagger}$ represents the cavity without injection and $M^{\ddagger}$ represents the cavity with injection. The two types of models are treated slightly differently, the implementation are illustrated in Appendix B. Our objective is to construct DAG $G = (V, E)$ for the corresponding input model. Each node V in G represents a contour unit after slicing the input model, and each directed edge E encodes a dependency between nodes.

The graph G is composed of three subgraphs: G_H, G_C, and G_I, corresponding to the height dependency constraint, collision-free constraint, and injection dependency constraint, respectively, as introduced in subsection 5.1. The construction details of G_H and G_C are provided in Appendix A, with the schematic illustration in Figure 8 and Figure 9. While the generation of the G_I graph is described as follows.

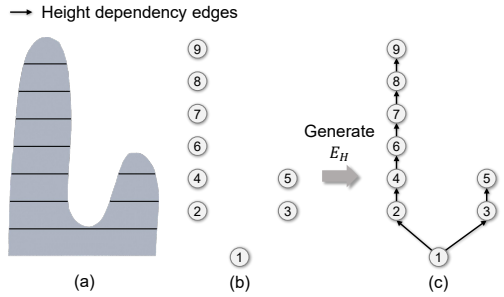


Figure 8: Generation of the height dependency graph. (a) Slice the input model M to obtain contour layers. (b) Construct the graph nodes V based on the resulting contours c. (c) Establish the height dependency edges E_H between nodes according to their vertical relationships in c.

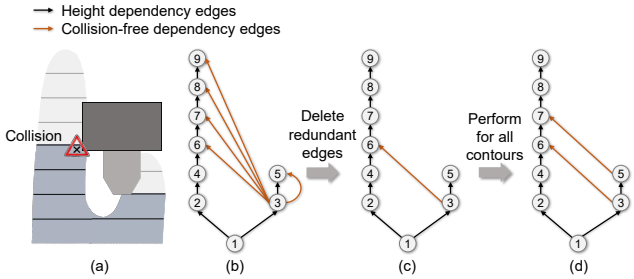
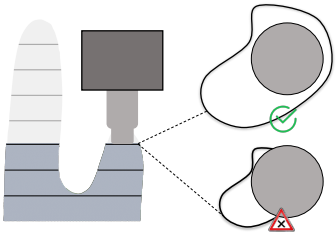


Figure 9: Collision detection and construction of the collision-free dependency graph. (a) Traverse contour 3 to detect potential collisions with other contours. (b) Contour 3 is found to collide with five other contours, resulting in the addition of five collision-free dependency edges. (c) Redundant edges are removed, retaining only the most critical one. (d) This process is repeated for all contours, ultimately yielding two valid collision-free dependency edges in the final graph.



Injection dependency graph. For the injection dependency between contours, we first identify the set of switching contours (i.e., the contours at which the printing nozzle switches to injection during fabrication), denoted as $\mathbf{c}^*$, and then establish injection dependency edges based on $\mathbf{c}^*$. The specific process is as follows:

As an switching contour, two conditions must be met: (1) the contour must be located at the top of a cavity region within M, ensuring that the injected material fills the entire cavity; (2) the **Maximum Inscribed Circle (MIC)** of the contour must have a diameter greater than or equal to the injection nozzle diameter $d_{injection}$ to prevent material leakage during injection.

The G_H can reflect the geometric features of M. The nodes with an out-degree of zero correspond to contours at the top of cavities. Therefore, we begin identify the contours in G_H that correspond

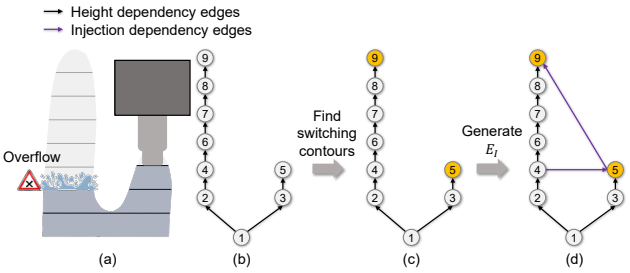


Figure 10: Generation of the injection dependency graph. (a) An example when an injection failure occurs. (b) The height dependency graph G_H of M. (c) Starting from the leaf nodes of G_H, identify the switching contour $\mathbf{c}^*$ whose feature size matches the injection nozzle diameter $d_{\text{injection}}$. (d) Generate injection dependency edges E_I according to the injection dependency constraints.

to these zero-degree nodes. and evaluate whether their MIC meets $d_{injection}$ requirement. Specifically, we generate a set of uniformly distributed discrete points $\mathbf{v} = \{v_1, v_2, \ldots, v_k\}$ within the candidate contour. Then, we apply the **Euclidean Distance Transform (EDT)** to compute the shortest distance from each discrete point to the contour boundary:

$$\xi(v) = \min_{c_i \in \mathbf{c}} \|v - c_i\|, \tag{2}$$

where $\xi(v)$ represents the shortest Euclidean distance from to candidate contour c_i. The center v^* of the MIC is determined by the point with the largest distance value (note that v^* can be used as the positioning coordinates of the injection nozzle):

$$v^* = \arg\max_{v \in \mathbf{v}} \xi(v), \tag{3}$$

the corresponding radius r^* is given by:

$$r^* = \max_{v \in \mathbf{v}} \xi(v). \tag{4}$$

After obtaining the radius r^* of the MIC for the candidate contour c_i, we determine whether it qualifies as an switching contour by evaluating the relationship between r^* and $d_{injection}$. If $2 \cdot r^* \geq d_{injection}$, the c_i is confirmed as the switching contour c_i^*. Otherwise, we retrieve the contour from G_H corresponding to its preorder node and repeat the computation for the new candidate contours. The final identified switching contours are shown as the yellow nodes in Figure 10(c).

After identifying all switching contours $\mathbf{c}^*$, we construct the injection dependency graph G_I. A directed edge is created in G_I under the following conditions: (1) if a given contour has the same height as an switching contour, a dependency edge is established; (2) if a contour itself is an switching contour, dependency edges are added from it to all other switching contours with a greater height. Mathematically, this relationship is defined as follows:

$$G_I = \begin{cases} 1 & \text{if } z(c_i) = z(c_j^*) \text{ or } z(c_i^*) \leq z(c_j^*), \\ 0 & \text{otherwise.} \end{cases} \tag{5}$$

The resulting G_I is shown in Figure 10(d), which will be used for subsequent sequence planning.

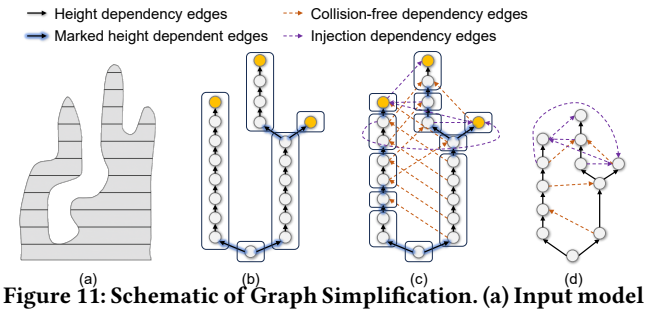


Figure 11: Schematic of Graph Simplification. (a) Input model M. (b) Mark the dependent edges according to the degrees of the nodes in G_H. (c) Further mark the dependent edges in G_H in combination with graph $G_{C,I}$. (d) Traverse G_H using the DFS algorithm, and merge the nodes before encountering the marked edges into a subgraph, serving as a new node V^*.

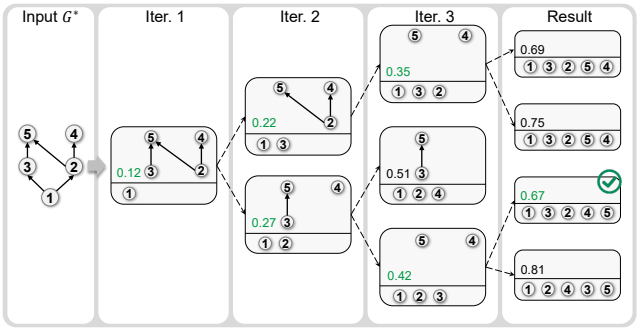


Figure 12: An example of our beam search algorithm. We set the beam width $\mathcal{W} = 2$ in this demo. In each iteration, new paths are generated based on the nodes with an in-degree of 0. When the number of branches exceeds $\mathcal{W}$, pruning is carried out according to the scoring function $f(\cdot)$. In the final iteration, the solutions with the highest score is selected as the final solution.

5.3 Graph simplification

Although the dependency graph G can directly used to finds an optimal solution (in subsection 5.4), the running time grows rapidly with the number of contours. In this section, we propose a heuristic simplification algorithm that enables reduction in the number of contour sets of consider, and allow us to speed up the solution substantially. The algorithm first marks certain dependency edges in G_H. Then, a **Depth-First Search (DFS)** is performed on G_H, grouping all nodes before encountering a marked edge into a subgraph. This subgraph is then treated as single new node, effectively reducing the size of the graph. Finally we introduce a parameter σ that allows to partition the subgraphs after simplification to control the complexity. The detailed procedure is described in Appendix D and Figure 11.

5.4 Shortest sequence exploration

We can directly apply topological sorting in G^* to determine the manufacturing sequence. In this approach, whenever multiple nodes have an in-degree of zero, a new branch is created, resulting in n possible solutions. The optimal solution can then be selected from these candidates. While this method guarantees a globally optimal solution, it comes at the cost of increased space and time complexity.

An alternative, more efficient approach avoids branching by heuristically selecting the next node based on optimization objectives. Although this method significantly reduces computational overhead and quickly produces a feasible solution, it does not guarantee optimality.

To strike a balance between solution quality and efficiency, we adopt a **beam search** algorithm. This approach retains multiple promising solution paths while keeping the search space manageable. Appendix E outlines the core procedure of the beam search algorithm, with the workflow illustrated in Figure 12.

The final manufactured ordering is the sequence of subgraphs obtained by beam search. Within each subgraph, the contours are sorted according to their dependencies.

6 POST-PROCESSING

During 3D printing process, two common issues may arise: interlayer gaps and collapses due to lack of internal support (see Figure 13(a)). These issues stem from the fundamental principle of 3D printing, where material layers are sequentially stacked to fabricate a three-dimensional object. Each layer requires a stable foundation built upon the previous one. However, during the slicing process, when the surface inclination angle (defined as the angle between the surface tangent and the $X-Y$ plane) is too small, interlayer gaps may form.

In previous research, such interlayer gaps were often considered negligible, as their impact on print quality was minimal [57]. However, in our study, these gaps pose significant challenges. During the casting process, these defects can lead to leakage, and in severe cases, result in manufacturing failure. Similarly, collapses due to lack of internal support occur when the unsupported area beneath the current deposition contour is too large, leading to structural instability. Essentially, this issue is an extreme case of interlayer gaps, where an entire section remains unsupported, causing collapse.

To address these issue, we propose a post-process method to eliminates interlayer gaps. Specifically, we traverse all contours $\mathbf{c}$ from top to bottom based on height and compute the filled regions R_i in c_i using the following formula:

$$R_i = \left(\mathcal{L}'_{i+1} - c_i\right) \cup \left(\mathcal{L}_{i-1} - c_i\right). \tag{6}$$

Here, $\mathcal{L}_{i+1}$ represents the set of all contours where c_j satisfies $z(c_j) - z(c_i) = h$ and $(c_i, c_j) \in G_H$. Similarly, $\mathcal{L}_{i-1}$ represents the set of all contours where c_j satisfies $z(c_i) - z(c_j) = h$ and $(c_j, c_i) \in G_H$. $\mathcal{L}'_i$ is a transformed contour set proposed to satisfy internal support requirements, c'_i is defined as:

$$c'_i = \text{offset}(\mathcal{L}'_{i+1} \cap c_i, \delta). \tag{7}$$

Here, δ is the offset distance that controls the slope of the internal support, usually equal to a path width w.

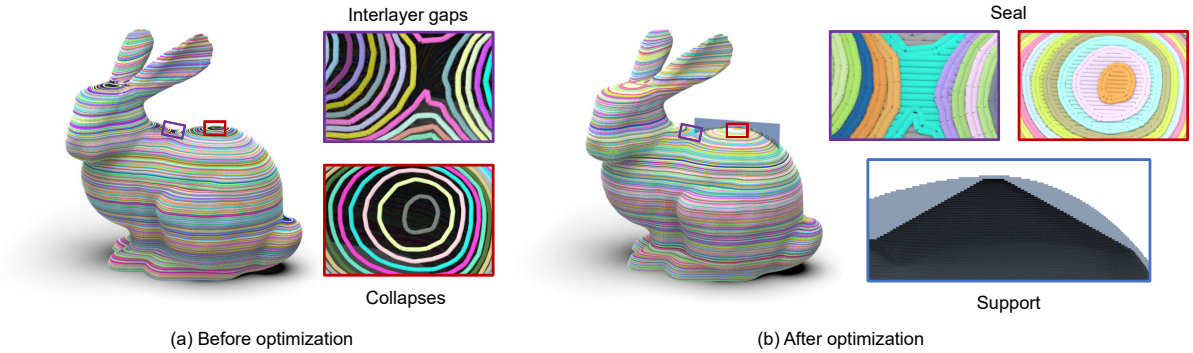


Figure 13: Comparison of interlayer gaps optimization. (a) The result before optimization exhibits noticeable interlayer gaps and overhanging contours, which would lead to collapse during actual printing. (b) After optimization, the interlayer gaps have been effectively filled, and the addition of internal support prevents collapse.

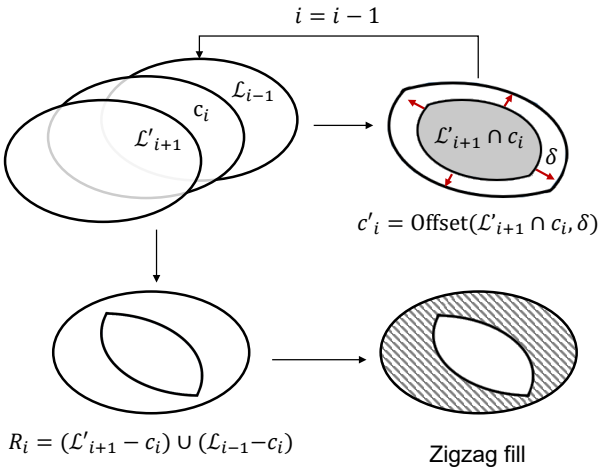


Figure 14: Schematic of interlayer gaps optimization. A top-down approach is adopted to eliminate interlayer gaps. For each contour c_i, we compute its difference with adjacent layer contours $\mathcal{L}'_{i+1}$ and $\mathcal{L}'_{i-1}$, generating a residual region R_i filled with a zigzag pattern. Meanwhile, the intersection of c_i and $\mathcal{L}'_{i+1}$ is offset to produce a new contour c'_i, which replaces the original. Repeating this process layer by layer ensures internal support and reduces unsupported regions.

Finally, we fill R with a Zigzag path to ensure sealing. Similarly, for each switching contour $\mathbf{c}^*$, after the injection process is completed, it will also be filled to prevent leakage. The details of the algorithm are referenced as shown in Figure 14, and the results as shown in Figure 13(b). Utilizing the optimized contours, we ascertain the necessary liquid volume for each injection phase (refer to Appendix F).

7 USER INTERFACE OF INFUSE

We have developed a user-friendly interface built upon the proposed algorithm, tailored for users with prior experience in 3D modeling and manufacturing. Implemented in C++ on the Windows platform, the interface leverages the libigl [28] and ImGui [15] libraries to support interactive visualization. The software accepts pre-designed models as input and enables users to intuitively control the manufacturing outcome through a graphical user interface. The interface consists of five main functional modules: model import, injection cavity selection, parameter configuration, printing path generation, and result preview. These components are organized in a top-down workflow, offering a streamlined and user-centric experience. Figure 15 illustrates the details of the interface.

Model input. This is the starting point of the workflow. On the interface, users begin by specifying the type of model—either a single-cavity or multi-cavity model. Once the type is selected, the model can be loaded from the local system using the "Load" button.

Injection Cavity Selection. If a single-cavity model is loaded, the system automatically highlights the cavity for easy identification. For multi-cavity models, users can manually select the desired injection cavity. The selected cavity will be visually emphasized, improving recognition and interaction.

Parameter Configuration. This module allows dynamic adjustment of key parameters. These include nozzle-specific parameters (such as height, length, and width; see Supplementary File 1 for details), printing parameters (e.g., layer thickness, path width, wall thickness, printing speed), material selection (including the option for hybrid materials), as well as mixing ratios and flow rates for hybrid configurations.

Printing Path Generation. After setting the parameters, users can initiate path planning by clicking the "Do" button. During computation, a progress bar provides real-time feedback, allowing users to track the generation process visually.

Result Preview. Once the printing path is successfully generated, the system displays relevant result parameters. A built-in slider enables users to preview the slicing results layer by layer. Additionally, quantitative information about the printing can be reviewed in the interface.

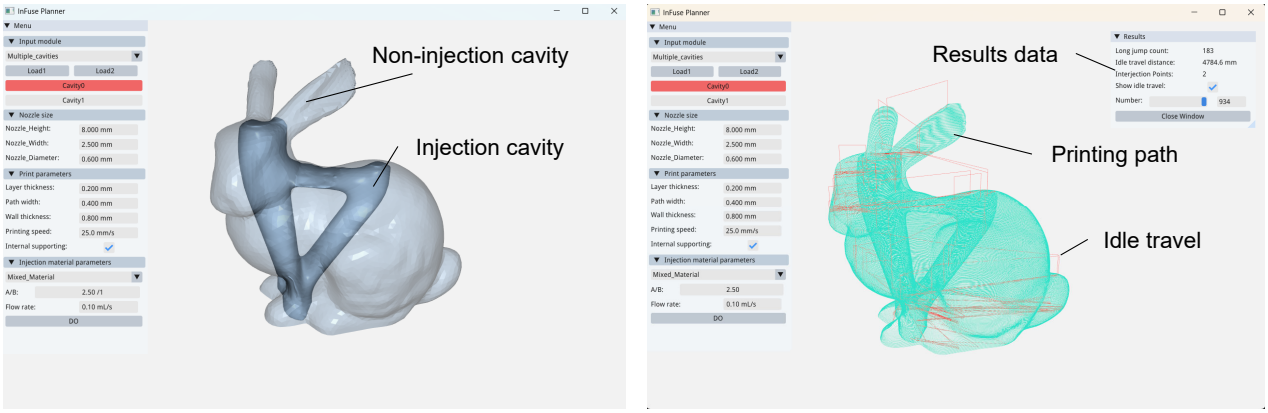


Figure 15: The user interface. (a) The control panel enables users to customize manufacturing outcomes by adjusting key parameters. (b) The result visualization interface provides an intuitive view of the printing path, nozzle travel trajectory, and output data.

This structured and interactive design helps streamline the workflow and empowers users to efficiently fine-tune and preview their hybrid manufacturing process.

8 RESULTS AND DISCUSSIONS

This section shows the InFuse's outcomes for 3D models with different geometric complexities and their physical assessments. We also evaluate our optimization algorithm's performance across various parameter settings.

8.1 Implementation and parameters

The implementation of our sequence planning algorithm was carried out in C++, utilizing a PC equipped with an Intel Core i7-13700F CPU operating at 2.5GHz and 32GB of memory. Unless otherwise specified, we use a slicing layer thickness h of 0.2 mm, a simplification degree Γ of 1, a beam width $\mathcal{W}$ of 7500, and a scoring function weight factor μ of 0.5. The internal support slope δ is set to a value equal to the path width of 0.4 mm. The peripheral diameter of the injection nozzle is 2.5 mm. Further details on the 3D printing nozzle parameters can be found in Appendix C. The fabrication experiments were conducted using in-situ material injection 3D printing platform, as detailed in the section 4. A demonstration video of the manufacturing process accompanies this paper.

8.2 Calculation results

In Figure 16, we present the simulated results of 7 different models, which encompass both single-cavity models, and multi-cavity models. The model we used with topology optimization results was generated by abaqus software [47]. Meanwhile, we also list the statistical data from Table 1 and the running times from Table 2 to evaluate the performance of our algorithm.

Performance. We have verified the robustness of our algorithm through a large number of models, encompassing single-cavity models with complex geometries and lightweight multi-cavity models derived from topology optimization. As can be seen from Table 1,

Table 1: Statistics of the models shown in the paper. H represents the height of M after its setup is determined. $\#P_I$ denotes the number of injection point in M. $\#V$ is the number of nodes in the dependency graph G. $\#E_H$, $\#E_C$, and $\#E_I$ indicates the number of height dependency edges, collision-free dependency edges, and injection dependency edges, respectively. $\#E$ represents the total number of the edges in G. $\#V^*$ and $\#E^*$ denote the number of nodes and edges in the simplified graph G^*, respectively. D_T refers to the idle travel distance of the printhead after sequence optimization, with J_N representing the corresponding number of long-jumps.

Input		Sec. 4.1						Sec. 4.2		Sec. 4.3	
Model	H	$\#P_I$	$\#V$	$\#E_H$	$\#E_C$	$\#E_I$	$\#E$	$\#V^*$	$\#E^*$	D_T	$\#J_N$
Pipe	62	3	791	790	836	6	1632	125	368	816.7	17
Hand	120	5	1360	1359	2644	20	4023	117	632	968.1	25
Kitten	78	3	475	475	66	5	546	37	62	93.829	5
Bunny	78	4	472	471	79	8	558	26	44	149.1	5
Julia vase	80	1	778	780	1024	0	1804	48	212	301.6	12
Kitten-topo	78	1	830	829	858	0	1687	352	839	4964.1	193
Bunny-topo	78	2	834	835	1204	2	2141	444	1247	4784.6	183

our graph simplification method can reduce the vertices of the graph of single-cavity models by up to 95% and the dependent edges by up to 93%. In contrast, for multi-cavity models, the vertices can only be reduced by up to 58% at most, and the dependent edges by 51%.

Likewise, the idle travel distance of the print nozzle and long-jumps also conform to this characteristic. This is mainly attributed to the intricate collision scenarios that occur in multi-cavity models. The complex spatial structures and arrangements within multi-cavity models result in more challenging collision avoidance requirements, which consequently influence the reduction rates of vertices and edges in the graph. However, our algorithm has shown its capability to handle such complex situations effectively,



Figure 16: Gallery of results generated using InFuse planner. The displayed models include *Bunny-topo*, *Kitten*, *Julia vase*, *Kitten-topo*, *Bunny*, *Hand*, and *Pipe*. The injection nozzle is also shown. From left to right, the images represent different stages of the manufacturing process. It can be observed that our approach effectively prevents air cavity issues while seamlessly integrating the beyond layer algorithm into InFuse.

Table 2: Program running time of each step (in seconds). T_G represents the time required to generate the dependency graph, T_{G^*} denotes the time spent on graph simplification, T_B corresponds to the execution time of our beam search method, T_P is the time taken for post-processing, and T represents the total running time.

Model	T_G	T_{G^*}	T_B	T_P	T
Pipe	1.745	0.001	29.405	349.015	380.166
Hand	0.714	0.004	49.041	382.91	432.669
Kitten	12.8	0.001	5.123	162.863	180.787
Bunny	56.315	0.001	1.033	359.489	416.838
Julia vase	41.435	0.001	13.591	214.553	269.58
Kitten-topo	0.725	0.004	77.167	50.73	128.626
Bunny-topo	2.364	0.002	144	162.556	308.922

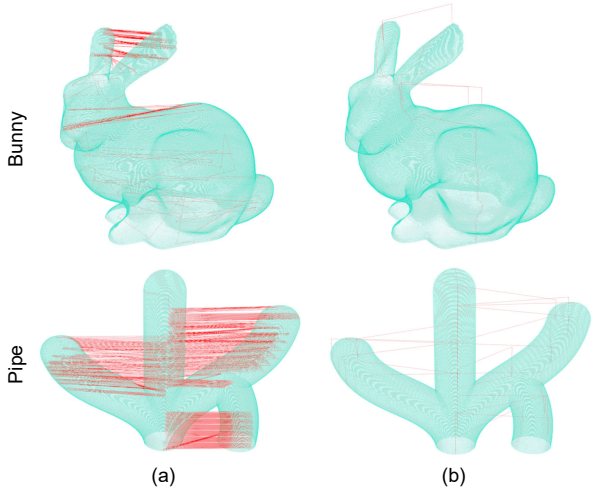


Figure 17: Comparison with traditional layer-by-layer. (a) Traditional layer-by-layer fabrication method. (b) InFuse fabrication method. The first row shows the *bunny* case, and the second row shows the *pipe* case. Cyan lines represent the printing paths, while red lines indicate idle travel. It is evident that InFuse achieves shorter idle travel distances and fewer long-jump count.

as indicated by the performance metrics in the reduction of graph elements and the results of the print head's movement.

Runtime. Table 2 illustrates the running times of our algorithm when applied to different models. It is evident that there is a significant variation in the time required for generating the dependency graph G of each model. This is primarily because the majority of the time in this stage is consumed by the generation of collision-free dependency edges. The more collision-free dependency edges that need to be detected in a model, the more time it will take. Although Table 1 shows that the number of collision-free dependency edges after removing redundant edges for these models is not large.

The time required for graph simplification is the shortest and can be almost negligible. This is due to the fact that our graph

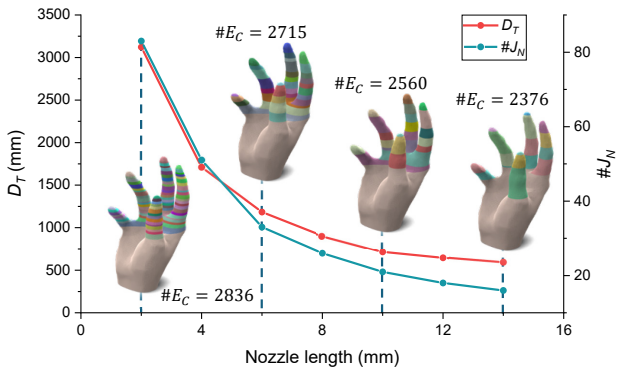


Figure 18: The *hand* model illustrates the results generated by our method, showing the relationship between idle travel distance D_T (left vertical axis), jump count $\#J_N$ (right vertical axis), and nozzle length (horizontal axis). Additionally, the figure provides the number of collision-free dependency edges $\#E_C$ and visualizes the results, where different colors in the model indicate nozzle long-jumps.

simplification method is heuristic. It only requires a traversal to mark the height dependency edges and then uses the DFS algorithm to generate subgraphs.

The time required for our beam search algorithm based on topology optimization entirely depends on the complexity of the graph G^* after graph simplification. As can be seen from Table 1 in combination, the higher the complexity of graph G^*, the more time is needed for this part.

The final post-processing stage mainly depends on the degree of change between the contours. When there are few interlayer gaps in the model, little time is needed for optimization.

In general, our algorithm can complete the calculation for most models within 10 minutes.

Comparison with traditional layer-by-layer. We use the traditional layer-by-layer 3D printing method as the baseline for algorithm comparison. As shown in Figure 17, the light cyan lines represent the printing paths, while the red lines indicate idle travel movements. It can be clearly observed that our algorithm significantly reduces both the frequency and distance of long-range empty travels.

8.3 Analysis of optimization parameters

We use the *hand* model as a demonstration example and study how main parameters affect 3D printing nozzle's idle travel distance ,the number of long-jump, and runtime by comparing various configurations. Key parameters are 3D printing nozzle length, degree of simplification Γ, beam width $\mathcal{W}$, and weighting factors μ for scoring function.

Effect of printing nozzle length. 3D printing nozzle length directly affects collision-free dependency edges generation and the final optimization results. Shorter nozzles collide more frequently with the model, while longer nozzles reduce collisions, leading to better results.

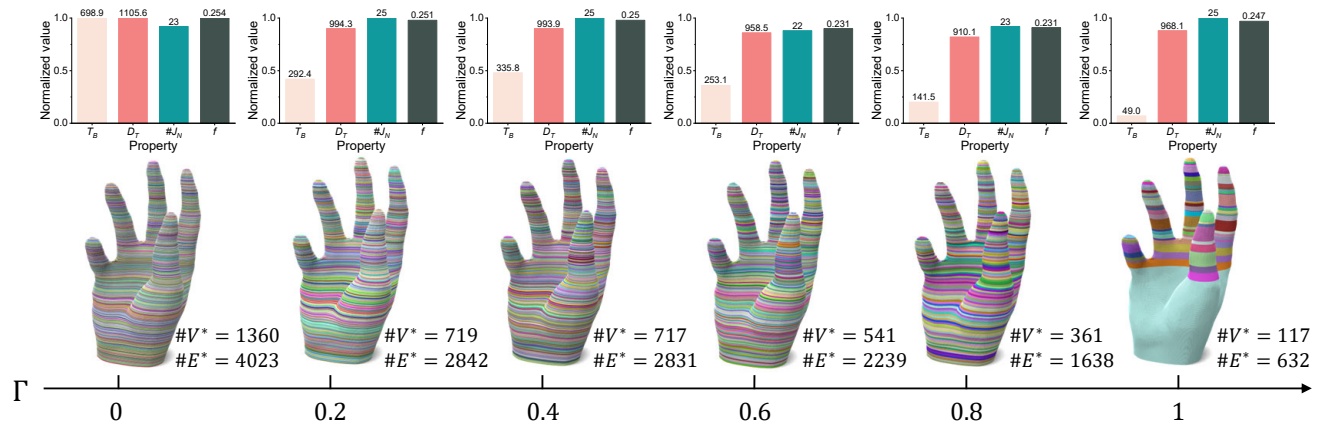


Figure 19: The impact of graph simplification. From left to right, the results correspond to a simplification degree Γ ranging from 0 to 1. The first row presents statistical results of the property, while the second row provides visualizations (different colors indicate the subgraphs after simplification). The figure includes the beam search runtime T_B (in seconds), idle travel distance D_T (in mm), jump count $\#J_N$, scoring function f, number of simplified vertices $\#V^*$, and number of simplified edges $\#E^*$.

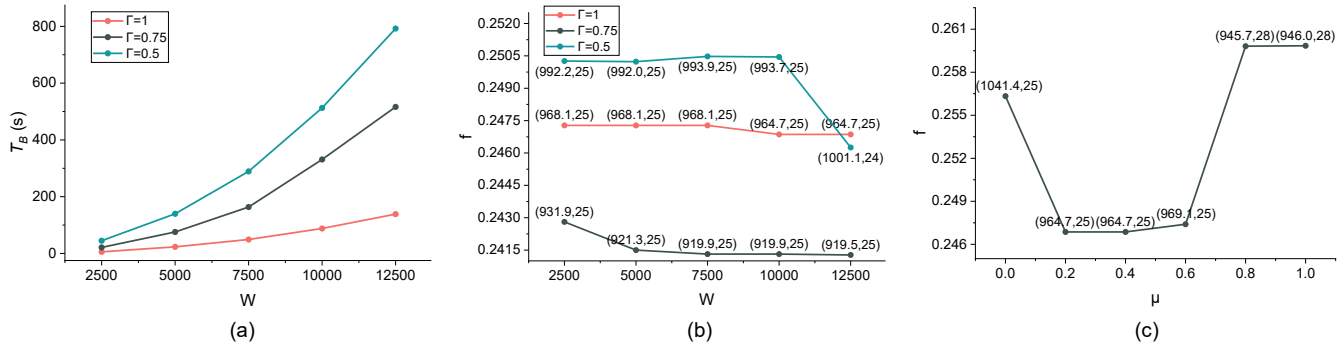


Figure 20: The impact of beam search parameters on the results is illustrated. (a) and (b) show the effect of Beam width $\mathcal{W}$ under different graph simplification degrees Γ. Specifically, (a) presents the statistical results of runtime T_B with $\mathcal{W}$, while (b) shows its effect on the scoring function, with each node annotated with idle travel distance D_T and jump count $\#J_N$. (c) illustrates the influence of scoring function weights μ on the final score, with each node also labeled with D_T and $\#J_N$.

We conducted experiments with nozzle lengths from 2 mm to 16 mm. The results (see Figure 18) confirm that increasing nozzle length reduces collision-free dependency edges, idle travel distance, and long-jump counts, demonstrating that a longer nozzle improves performance.

Effect of degree of simplification. The degree of simplification directly influences graph size and computational efficiency. Figure 19 shows the results of our tests for different Γ. As simplification increases, the number of vertices $\#V^*$ and edges $\#E^*$ decreases, leading to shorter Beam search runtime T_B. This reduction helps improve efficiency, as a smaller graph allows for faster exploration of potential solutions.

However, the results show a non-monotonic trend in the evaluation score f. Initially, simplification improves the score by simplifying the graph and reducing unnecessary complexity. But as simplification increases further, the score starts to rise again, suggesting that excessive graph reduction may lead to suboptimal solutions. This trade-off indicates that while simplification can enhance computational feasibility, it may also cause a loss of critical information, potentially moving the solution further from the true optimum.

Effect of beam width. As illustrated in Figure 20(a) and (b), we have statistical the results corresponding to varying beam widths $\mathcal{W}$ under different Γ. The results demonstrate that increasing beam width $\mathcal{W}$ generally improves the quality of the solutions, as indicated by the decrease in the evaluation function f. However, this improvement comes at the cost of increased runtime T_B. This is because a wider beam width enables the algorithm to explore more solutions, thus improving the optimization results.

However, in cases with Γ close to 1, the improvement levels off, and performance stabilizes even with further increases in $\mathcal{W}$. This implies that in highly simplified graphs, there are limited solutions to explore, so a wider beam width offers little extra benefit. Conversely, at lower simplification degrees, while larger beam widths initially improve results, there comes a point where performance deteriorates. This is due to factors such as the beam search exploring suboptimal paths because of early pruning or getting trapped in local optima as the search space expands with lower simplification.

In summary, there is a diminishing return with increasing beam width, with the most significant improvements at moderate $\mathcal{W}$ values. In more complex graphs, excessive beam width can lead to over-exploration, degrading solution quality due to search biases or being unable to escape local minima. Thus, while larger beam widths boost solution accuracy, they must be balanced with problem size and complexity.

Effect of weighting factor. From Figure 20(c), it is clear that when the weighting factor μ is either close to 0 or 1, the results are not as good as when μ takes on intermediate values. This suggests that when the algorithm focuses too strongly on either distance or long-jump, it is more likely to get stuck in local optima. In the beam search process, having a single, clear objective can lead the algorithm to converge prematurely on suboptimal solutions.

When μ is set to a value around 0.2 or 0.4, both distance and long-jump are considered, introducing a degree of "perturbation" in the search process. This helps the algorithm avoid being too narrowly focused, allowing it to explore a broader set of solutions and ultimately find a better overall solution.

Additionally, introducing the parameter μ allows our algorithm to be extended to a broader range of applications. For example, printing carbon fiber or ceramic materials may require minimizing the number of long-jumps, whereas materials like PLA or ABS can focus more on reducing idle travel distance.

8.4 Physical evaluation

We selected *bunny* and *pipe* models from Figure 16 to demonstrate the physical fabrication process. As shown in Figure 21, compared to the conventional "printing-then-injection" approach, InFuse enables the construction of more complex structures without requiring manual sealing or the addition of auxiliary flow channels.

Furthermore, we evaluate the fabrication efficiency of InFuse against full solid printing using 3DP alone (the contour printing speed is 20 mm/s and the infill printing speed is 40 mm/s). As shown in Figure 22, InFuse achieves up to a 70% reduction in total fabrication time. The improvement is more significant for larger models. Owing to its injection nature, the fabricated parts also exhibit isotropic mechanical properties, which address the inherent anisotropy commonly observed in traditional 3DP processes.

However, since the injection process is fully automated, discrepancies between the theoretically computed liquid volume (as discussed in Appendix F) and the actual injected amount may result in minor voids within the internal cavities, as illustrated in Figure 21. Additionally, for contours with dimensions smaller than the injection nozzle diameter $d_{injection}$, a zigzag infill pattern is employed to ensure complete material filling. In the *pipe* model, relatively thin wall structures are deliberately used; while a slight

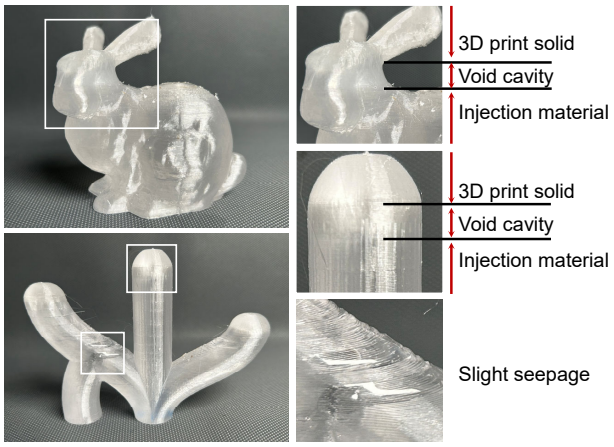


Figure 21: Physical evaluation of the fabricated models. The left side shows the physical fabrication results of the *bunny* and *pipe* models using the InFuse system. The right side provides magnified views highlighting representative structural details, along with minor imperfections caused by parameter deviations and fabrication tolerances.

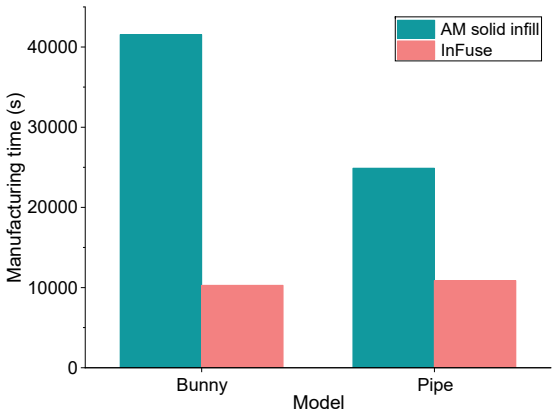


Figure 22: Comparison of total fabrication time between InFuse and traditional AM solid infill methods for the *bunny* and *pipe* models. InFuse demonstrates significant time savings, with larger models exhibiting greater improvements.

amount of material seepage may occur over time. These phenomena arise from practical deviations inherent in the fabrication process and can be effectively mitigated through appropriate parameter tuning by the user.

9 APPLICATIONS

With its capacity for complex geometry fabrication and broad material compatibility, InFuse offers significant advantages across a variety of application domains. The system supports rapid fabrication of monolithic structures with improved isotropic mechanical properties, thereby addressing key limitations in conventional 3D printing. These advantages enable its use in a variety of scenarios,

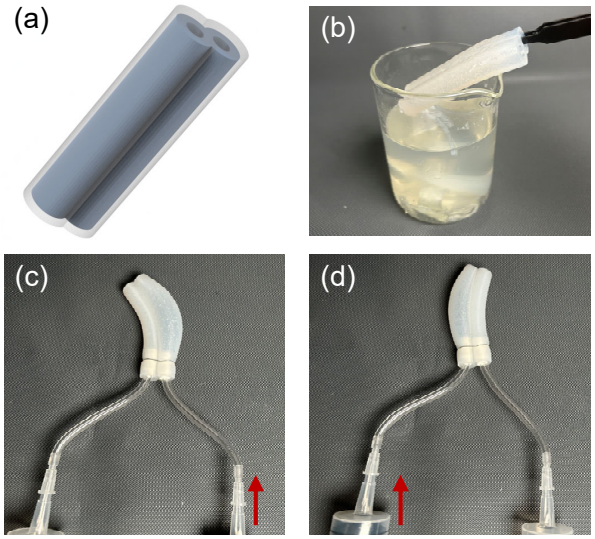


Figure 23: Pneumatic soft actuator fabricated using InFuse. (a) The actuator structure. (b) Final actuator obtained after dissolving the sacrificial PVA mold. (c) When air is injected into the right chamber, the actuator bends to the left. (d) Conversely, injecting air into the left chamber causes the actuator to bend to the right.

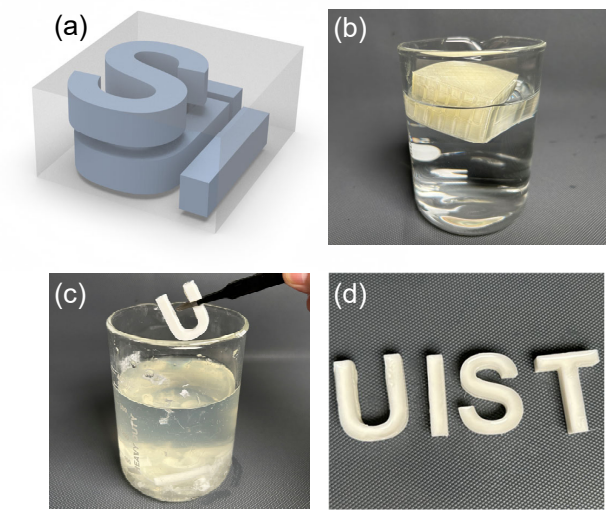


Figure 24: Demonstration of sacrificial mold manufacturing using InFuse. (a) Multiple models arranged using a packing algorithm for compact layout. (b) Fabricated PVA mold with internal cavities already filled with injected material. (c) Mold after partial dissolution in water. (d) Extracted internal models following complete mold removal.

including the fabrication of soft-bodied robots, artistic and architectural model creation, rapid mold prototyping for casting, and biomedical structures with embedded fluidic pathways. In addition, the ability to encapsulate functional materials during the printing process makes InFuse a promising solution for the integration of embedded electronics and multi-material functional devices. In the following, we demonstrate InFuse's versatility through representative examples across several of these domains.

9.1 Soft robotics

To explore the applicability of InFuse in the field of soft robotics, we fabricated a simple pneumatic soft actuator using a two-stage hybrid manufacturing process (see Figure 23(a)). The model consists of two internally connected cylindrical cavities enclosed in a soft elastomeric shell.

First, we used InFuse to print a sacrificial outer mold with water-soluble PVA, followed by injecting silicone resin into the internal cavities. After the resin was fully cured, the outer mold was dissolved in water, resulting in a flexible, hollow structure suitable for pneumatic actuation (see Figure 23(b)).

To demonstrate actuation behavior, air was manually injected into each cavity using a syringe. When one cavity was inflated, it expanded and caused the entire structure to bend in the opposite direction. Reversing the inflation direction caused the structure to bend the other way (see Figure 23(c) and (d)). This behavior showcases a fundamental motion mechanism widely used in pneumatic soft robots.

The experiment highlights InFuse's capability to fabricate deformable structures with air chambers—an essential feature in the development of soft robotic components.

9.2 Artistic fabrication

InFuse also demonstrates strong potential in the creation of artistic and design-oriented objects. Beyond producing visually compelling and geometrically intricate forms, it supports the fabrication of playful and imaginative creations. As shown in Figure 1(d) and (e), we used semi-transparent PETG as the outer shell material and filled the internal cavities with a UV-curable, fluorescent adhesive. The resulting structure is fully sealed and contains multiple non-interconnected internal cavities. InFuse accommodates this fabrication seamlessly, enabling the production of objects that are not only visually striking but also structurally meaningful.

9.3 Sacrificial mold fabrication

InFuse further extends its applicability to 3D printed mold fabrication. Notably, it supports the production of sacrificial family molds without the need for flow channels. In our demonstration, we employed water-soluble PVA as the mold material and epoxy resin for casting. A set of models was generated and arranged using a packing algorithm to maximize spatial efficiency, as illustrated in Figure 24(a). Non-injection cavities were internally supported using grid structures. The fabricated results are shown in Figure 24(d). Compared to conventional methods, our approach benefits from the ability to form non-interconnected cavities, which removes the constraint of pre-designed flow channels. This enables significantly tighter packing of parts and reduces overall material consumption.

10 CONCLUSION AND FUTURE WORK

This paper introduces InFuse, the first fully automated hybrid manufacturing system that seamlessly integrates in-situ material injection with 3D printing. It achieves this by combining a computational pipeline and a customized hardware solution. The core of the computational pipeline is a DAG-based algorithm, which generates the hybrid manufacturing sequence for 3D printing and material injection. Moreover, the modified 3D printer can smoothly switch between printing and injection modes.

Despite the significant advancements achieved with InFuse, there are still limitations to our approach. The current system's DAG-based computational pipeline may face challenges when dealing with extremely complex 3D models with highly irregular internal structures. The modified 3D printer hardware has limited accuracy in controlling the volume of injected material. Hence, the in-situ material injection process may not be suitable for some applications where very high precision in the distribution of the injected material is crucial. The current injection mechanism may not be compatible with certain high-performance or specialized materials. This incompatibility restricts the full range of applications in which InFuse can be deployed. In addition, our interactive interface does not involve the design of internal injection channels, which to some extent restricts the application scope of the system.

For future work, we will enhance the computational efficiency of the DAG-based algorithm using advanced optimization techniques and machine learning to predict and optimize manufacturing sequences. Another direction is developing a multi-axis platform and its computational pipeline, enabling more flexible movement during 3D printing and material injection processes for complex geometries. On the hardware side, we will focus on synchronizing 3D printing and material injection by setting asynchronous printing heads and injection nozzles, as well as expanding material compatibility to include high-performance polymers, metals, and composites. Furthermore, we will explore ways to enhance the precision of the in-situ material injection process by improve the control mechanisms that can precisely control the flow and distribution of the injected material. Finally, we will continue to explore new applications for InFuse, especially in emerging fields such as bio- manufacturing and micro-electromechanical systems (MEMS). By demonstrating the effectiveness of InFuse in these areas, we hope to further expand the impact and reach of our hybrid manufacturing technology.

REFERENCES

[1] Osama Abdulhameed, Abdulrahman Al-Ahmari, Wadea Ameen, and Syed Hammad Mian. 2019. Additive manufacturing: Challenges, trends, and applications. *Advances in Mechanical Engineering* 11, 2 (2019), 1687814018822880.

[2] Thomas Alderighi, Luigi Malomo, Thomas Auzinger, Bernd Bickel, Paolo Cignoni, and Nico Pietroni. 2022. State of the art in computational mould design. In *Computer Graphics Forum*, Vol. 41. Wiley Online Library, 435–452.

[3] Thomas Alderighi, Luigi Malomo, Bernd Bickel, Paolo Cignoni, and Nico Pietroni. 2021. Volume decomposition for two-piece rigid casting. *ACM Trans. Graph.* 40, 6 (2021), 272–1.

[4] Thomas Alderighi, Luigi Malomo, Daniela Giorgi, Bernd Bickel, Paolo Cignoni, and Nico Pietroni. 2019. Volume-aware design of composite molds. *ACM Transactions on Graphics* (2019).

[5] Thomas Alderighi, Luigi Malomo, Daniela Giorgi, Nico Pietroni, Bernd Bickel, and Paolo Cignoni. 2018. Metamolds: computational design of silicone molds. *ACM Transactions on Graphics (TOG)* 37, 4 (2018), 1–13.

[6] All3DP. 2025. *Fraunhofer IPA Combines 3D Printing and Injection Molding for New Advantages.* https://all3dp.com/fraunhofer-ipa-combine-3d-printing-injection-molding-advantages/ Accessed: 2025-04-09.

[7] Lalatovic Andjela, Vaniev Marat Abdurahmanovich, Sidorenko Nina Vladimirovna, Gres Irina Mikhailovna, Dyachenko Denis Yurievich, and Makedonova Yulia Alekseevna. 2022. A review on Vat Photopolymerization 3D-printing processes for dental application. *Dental Materials* 38, 11 (2022), e284–e296.

[8] Joseph T Belter and Aaron M Dollar. 2015. Strengthening of 3D printed fused deposition manufactured parts using the fill compositing technique. *PloS one* 10, 4 (2015), e0122915. doi:10.1371/journal.pone.0122915

[9] Dustin Beyer, Serafima Gurevich, Stefanie Mueller, Hsiang-Ting Chen, and Patrick Baudisch. 2015. Platener: Low-fidelity fabrication of 3D objects by substituting 3D print with laser-cut plates. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*. 1799–1806.

[10] Li Chen, Ke Xu, and Kai Tang. 2018. Optimized sequence planning for multi-axis hybrid machining of complex geometries. *Computers & Graphics* 70 (feb 2018), 176–187. doi:10.1016/j.cag.2017.07.018

[11] Niechen Chen and Matthew Frank. 2019. Process planning for hybrid additive and subtractive manufacturing to integrate machining and directed energy deposition. *Procedia Manufacturing* 34 (2019), 205–213.

[12] Xiang'Anthony' Chen, Stelian Coros, Jennifer Mankoff, and Scott E Hudson. 2015. Encore: 3D printed augmentation of everyday objects with printed-over, affixed and interlocked attachments. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*. 73–82.

[13] Xuelin Chen, Honghua Li, Chi-Wing Fu, Hao Zhang, Daniel Cohen-Or, and Baoquan Chen. 2018. 3D fabrication with universal building blocks and pyramidal shells. *ACM Transactions on Graphics (TOG)* 37, 6 (2018), 1–15.

[14] Daniel L Cohen, Jeffrey I Lipton, Lawrence J Bonassar, and Hod Lipson. 2010. Additive manufacturing for in situ repair of osteochondral defects. *Biofabrication* 2, 3 (2010), 035004.

[15] Omar Cornut. 2025. ImGui. https://github.com/ocornut/imgui.

[16] Marc Crescenti, Irene Rafols, Antoni Lara, Milad Bemani, and Daniel Casellas. 2024. The continuous fibre injection process (CFIP): A novel approach to lightweight design of multi-material structural components. In *27th International ESAFORM Conference on Material Forming (ESAFORM 2024), Toulouse, France, April 24-26, 2024*. Materials Research Forum LLC, 1630–1639.

[17] Mohammadreza Lalegani Dezaki, Ahmad Serjouei, Ali Zolfagharian, Mohammad Fotouhi, Mahmoud Moradi, M.K.A. Ariffin, and Mahdi Bodaghi. 2022. A review on additive/subtractive hybrid manufacturing of directed energy deposition (DED) process. *Advanced Powder Materials* 1, 4 (oct 2022), 100054. doi:10.1016/j.apmate.2022.100054

[18] Ugur M Dilberoglu, Bahar Gharehpapagh, Ulas Yaman, and Melik Dolen. 2021. Current trends and research opportunities in hybrid additive manufacturing. *The International Journal of Advanced Manufacturing Technology* 113 (2021), 623–648.

[19] Ugur M. Dilberoglu, Bahar Gharehpapagh, Ulas Yaman, and Melik Dolen. 2021. Current trends and research opportunities in hybrid additive manufacturing. *The International Journal of Advanced Manufacturing Technology* 113, 3-4 (jan 2021), 623–648. doi:10.1007/s00170-021-06688-1

[20] Chad Duty, Jordan Failla, Seokpum Kim, Tyler Smith, John Lindahl, and Vlastimil Kunc. 2019. Z-Pinning approach for 3D printing mechanically isotropic materials. *Additive Manufacturing* 27 (2019), 175–184.

[21] Jimmy Etienne, Nicolas Ray, Daniele Panozzo, Samuel Hornus, Charlie CL Wang, Jonàs Martínez, Sara McMains, Marc Alexa, Brian Wyvill, and Sylvain Lefebvre. 2019. CurviSlicer: Slightly curved slicing for 3-axis printers. *ACM Transactions on Graphics (TOG)* 38, 4 (2019), 1–11.

[22] Thomas Feldhausen, Lauren Heinrich, Kyle Saleeby, Alan Burl, Brian Post, Eric MacDonald, Chris Saldana, and Lonnie Love. 2022. Review of Computer-Aided Manufacturing (CAM) strategies for hybrid directed energy deposition. *Additive Manufacturing* 56 (aug 2022), 102900. doi:10.1016/j.addma.2022.102900

[23] Thilo Feucht, Benedikt Waldschmitt, Jörg Lange, and Maren Erven. 2022. Additive manufacturing of a bridge in situ. *Steel Construction* 15, 2 (2022), 100–110.

[24] Joseph M. Flynn, Alborz Shokrani, Stephen T. Newman, and Vimal Dhokia. 2016. Hybrid additive and subtractive machine tools – Research and industrial developments. *International Journal of Machine Tools and Manufacture* 101 (feb 2016), 79–101. doi:10.1016/j.ijmachtools.2015.11.007

[25] Julien Gardan. 2017. Additive manufacturing technologies: state of the art and trends. *Additive Manufacturing Handbook* (2017), 149–168.

[26] Jaime Gould, Camila Friedman-Gerlicz, and Leah Buechley. 2024. TRAvel Slicer: Continuous Extrusion Toolpaths for 3D Printing. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–17.

[27] George P. Harabin and Morad Behandish. 2022. Hybrid Manufacturing Process Planning for Arbitrary Part and Tool Shapes. *Computer-Aided Design* 151 (oct 2022), 103299. doi:10.1016/j.cad.2022.103299

[28] Alec Jacobson, Daniele Panozzo, et al. 2018. libigl: A simple C++ geometry processing library. https://libigl.github.io/.

[29] Jingchao Jiang, Xiaoya Zhai, Liuchao Jin, Kang Zhang, Jun Chen, Qitao Lu, and Wei-Hsin Liao. 2023. Design for reversed additive manufacturing low-melting-point alloys. *Journal of Engineering Design* (2023), 1–14.

[30] Jingchao Jiang, Xiaoya Zhai, Kang Zhang, Liuchao Jin, Qitao Lu, Zhichao Shen, and Wei-Hsin Liao. 2023. Low-melting-point alloys integrated extrusion additive manufacturing. *Additive Manufacturing* 72 (2023), 103633.

[31] Daphna Kaplan, Shir Rorberg, Mirela Ben Chen, and Yoav Sterman. 2022. NozMod: nozzle modification for efficient FDM 3D printing. In *Proceedings of the 7th Annual ACM Symposium on Computational Fabrication*. 1–9.

[32] David O Kazmer and Austin Colon. 2020. Injection printing: Additive molding via shell material extrusion and filling. *Additive Manufacturing* 36 (2020), 101469.

[33] Marion Koelle, Madalina Nicolae, Aditya Shekhar Nittala, Marc Teyssier, and Jürgen Steimle. 2022. Prototyping soft devices with interactive bioplastics. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*. 1–16.

[34] John J Laureto and Joshua M Pearce. 2017. Open source multi-head 3D printer for polymer-metal composite component manufacturing. *Technologies* 5, 2 (2017), 36.

[35] Samuel Lensgraf and Ramgopal R Mettu. 2016. Beyond layers: A 3D-aware toolpath algorithm for fused filament fabrication. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 3625–3631.

[36] Samuel Lensgraf and Ramgopal R. Mettu. 2016. Beyond layers: A 3D-aware toolpath algorithm for fused filament fabrication. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. doi:10.1109/icra.2016.7487546

[37] Samuel Lensgraf and Ramgopal R. Mettu. 2017. An improved toolpath generation algorithm for fused filament fabrication. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. doi:10.1109/icra.2017.7989141

[38] Changqing Liu, Yingguang Li, Sen Jiang, Zhongyu Li, and Ke Xu. 2019. A sequence planning method for five-axis hybrid manufacturing of complex structural parts. *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture* 234, 3 (nov 2019), 421–430. doi:10.1177/0954405419883052

[39] Stefanie Mueller, Tobias Mohr, Kerstin Guenther, Johannes Frohnhofen, and Patrick Baudisch. 2014. faBrickation: fast 3D printing of functional objects by integrating construction kit building blocks. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 3827–3834.

[40] Sriram Pemmaraju and Steven Skiena. 2003. *Computational discrete mathematics: Combinatorics and graph theory with mathematica®*. Cambridge university press.

[41] Lan Ren, Kunnayut Eiamsa-ard, Jianzhong Ruan, and Frank Liou. 2007. Part repairing using a hybrid manufacturing system. In *International Manufacturing Science and Engineering Conference*, Vol. 42908. 1–8.

[42] Jihyuck Son, Seounghee Yun, Kundo Park, Seunghwa Ryu, and Sanha Kim. 2022. Isotropic 3D printing using material extrusion of thin shell and post-casting of reinforcement core. *Additive Manufacturing* 58 (2022), 102974.

[43] Peng Song, Bailin Deng, Ziqi Wang, Zhichao Dong, Wei Li, Chi-Wing Fu, and Ligang Liu. 2016. CofiFab: coarse-to-fine fabrication of large 3D objects. *ACM Transactions on Graphics (TOG)* 35, 4 (2016), 1–11.

[44] Evgeny Stemasov, Alexander Botner, Enrico Rukzio, and Jan Gugenheimer. 2022. Ephemeral Fabrication: Exploring a Ubiquitous Fabrication Scenario of Low-Effort, In-Situ Creation of Short-Lived Physical Artifacts. In *Proceedings of the Sixteenth International Conference on Tangible, Embedded, and Embodied Interaction*. 1–17.

[45] David Svetlizky, Mitun Das, Baolong Zheng, Alexandra L Vyatskikh, Susmita Bose, Amit Bandyopadhyay, Julie M Schoenung, Enrique J Lavernia, and Noam Eliaz. 2021. Directed energy deposition (DED) additive manufacturing: Physical characteristics, defects, challenges and applications. *Materials Today* 49 (2021), 271–295.

[46] Saiganesh Swaminathan, Kadri Bugra Ozutemiz, Carmel Majidi, and Scott E Hudson. 2019. Fiberwire: Embedding electronic function into 3d printed mechanically strong, lightweight carbon fiber composite objects. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–11.

[47] Dassault Systèmes. 2022. Abaqus Software. https://www.3ds.com/products-services/simulia/products/abaqus/. Computer software, Version 2022.

[48] Óscar Teixeira, Francisco JG Silva, Luís P Ferreira, and Eleonora Atzeni. 2020. A review of heat treatments on improving the quality and residual stresses of the Ti–6Al–4V parts produced by additive manufacturing. *Metals* 10, 8 (2020), 1006.

[49] DWJA van den Akker. 2017. The development of a hybrid manufacturing system that combines multimaterial 3D-printing with silicone casting for the development of soft robotic parts. *TUDelft Repository* (2017).

[50] Ludwig Wilhelm Wall, Alec Jacobson, Daniel Vogel, and Oliver Schneider. 2021. Scrappy: Using scrap material as infill to make fabrication more sustainable. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–12.

[51] Yushen Wang, Liang Hao, Yan Li, Qinglei Sun, Mingxi Sun, Yuhong Huang, Zheng Li, Danna Tang, Yijing Wang, and Long Xiao. 2022. In-situ utilization of regolith resource and future exploration of additive manufacturing for lunar/martian habitats: A review. *Applied Clay Science* 229 (2022), 106673.

[52] René Wick-Joliat, Maurice Tschamper, Roman Kontic, and Dirk Penner. 2021. Water-soluble sacrificial 3D printed molds for fast prototyping in ceramic injection molding. *Additive Manufacturing* 48 (2021), 102408.

[53] Wenpeng Xu, Hao Xu, Xiaoya Zhai, and Jingchao Jiang. 2024. Nature-inspired interlaced printing strategies for additive manufacturing highly improved mechanical properties. *Additive Manufacturing* 89 (2024), 104276.

[54] Xin Yan, Cong Rao, Lin Lu, Andrei Sharf, Haisen Zhao, and Baoquan Chen. 2019. Strong 3D printing by TPMS injection. *IEEE Transactions on Visualization and Computer Graphics* 26, 10 (2019), 3037–3050.

[55] Zeyu Yan, Hsuanling Lee, Liang He, and Huaishu Peng. 2023. 3D Printing Magnetophoretic Displays. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–12.

[56] Chanyeol Yoo, Samuel Lensgraf, Robert Fitch, Lee M. Clemon, and Ramgopal Mettu. 2020. Toward Optimal FDM Toolpath Planning with Monte Carlo Tree Search. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. doi:10.1109/icra40945.2020.9196945

[57] Fanchao Zhong, Yonglai Xu, Haisen Zhao, and Lin Lu. 2023. As-continuous-as-possible extrusion-based fabrication of surface models. *ACM Transactions on Graphics* 42, 3 (2023), 1–16.

[58] Fanchao Zhong, Haisen Zhao, Haochen Li, Xin Yan, Jikai Liu, Baoquan Chen, and Lin Lu. 2023. VASCO: Volume and Surface Co-Decomposition for Hybrid Manufacturing. *ACM Transactions on Graphics (TOG)* 42, 6 (2023), 1–17.

A BUILDING THE HEIGHT AND COLLISION-FREE DEPENDENCY GRAPH

Height dependency graph. For each height dependency between contours, two conditions must be satisfied: the contours must be adjacent in height, and their projections onto the xy-plane must intersect. Mathematically, this relationship is defined as follows:

$$G_H = \begin{cases} 1 & \text{if } \pi(c_i) \cap \pi(c_j) \neq \emptyset \text{ and } z(c_j) - z(c_i) = h, \\ 0 & \text{otherwise.} \end{cases} \tag{8}$$

Here, $\pi(\cdot)$ represents the projection of contour onto the xy-plane, while $z(\cdot)$ denotes the height of contour. The condition $z(c_j) - z(c_i) = h$ ensures that c_j is positioned exactly one layer above c_i, where h is the layer thickness. The result is shown in Figure 9(d) of the main text.

Collision-free dependency graph. For the collision-free dependency between contours, we introduce a nozzle $\mathcal{N}$ with a defined size (see Appendix C). During the traversal of c_i, we check whether $\mathcal{N}$ intersects with c_j. If an intersection occurs a collision-free dependency is established; otherwise, no dependency is required. This relationship is formulated as follows:

$$G_C = \begin{cases} 1 & \text{if } \mathcal{N}(c_i) \cap c_j \neq \emptyset, \\ 0 & \text{otherwise.} \end{cases} \tag{9}$$

Figure 9(b) of the main text shows the collision-free dependency edges generated by node ③ as a dependency. It is obvious that this method results in a large number of redundant edges. To address this, we maintain G as a Hasse diagram [40] in which redundant dependency edges do not exist. For example, if two edges $A \rightarrow B$ and $B \rightarrow C$ exist, then $A \rightarrow C$ is a redundant edge. After the above precessing, we remove the redundant edges in G_C as shown in Figure 9(c) of the main text. It is worth noting that the judgment of redundant edges occurs during the generation of all new edges, rather than being solely focused on the collision-free dependency edges. The final generated collision-free dependency graph is shown in Figure 9(d) of the main text.

B BUILDING THE DEPENDENCY GRAPH FOR SINGLE-CAVITY MODEL AND MULTI-CAVITY MODEL

The DAG generation can be applied to different types of input models. For a single-cavity model M, the dependency graph is constructed in the following order: first, the height dependency graph G_H is generated. Based on G_H, the collision-free dependency graph G_C and the injection dependency graph G_I are then constructed separately. Finally, the complete dependency graph is obtained as $G = (V, E_H \cup E_C \cup E_I)$.

For a multi-cavity model $M^\dagger$ and $M^\ddagger$, the corresponding height dependency graphs $G_H^\dagger$ and $G_H^\ddagger$ are first generated. The injection dependency graph $G_I^\ddagger$ is then constructed for $M^\ddagger$. These graphs are then merged to form $G_H = (V^\dagger \cup V^\ddagger, E_H^\dagger \cup E_H^\ddagger)$ and $G_I = (V^\dagger \cup V^\ddagger, E_I^\ddagger)$. Finally, the collision-free dependency graph G_C is generated for G_H, yielding the final dependency graph $G = (V^\dagger \cup V^\ddagger, E_H \cup E_C \cup E_I)$. The process is shown in Figure 25.

C NOZZLE SHAPE AND SIZE

The shape of our printing nozzle is illustrated in Figure 26. It is defined by four parameters: the total height H_a, the lower half height H_d (typically set to half of the total height), the nozzle diameter D_d, and the upper diameter D_u.

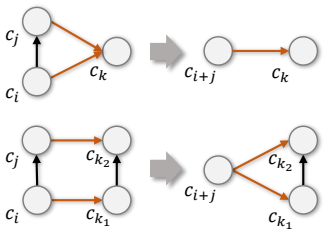
For collision detection, we determine whether the nozzle model intersects with any existing contour at the target position. A collision is identified if either an intersection occurs or if the height of the surrounding contour exceeds H_a.

D IMPLEMENTATION DETAILS GRAPH SIMPLIFICATION

Subgraph generation. First, we segment M at branching points based to its geometric shape using G_H. Specifically, we traverse each node in G_H and mark edges according to the following criteria: if a node has multiple direct predecessors $|P(V_i)| \geq 2$, all edges connecting this node to its direct predecessors are marked; similarly, if a node has multiple direct successors $|S(V_i)| \geq 2$, all edges connecting this node to its successors are marked. This operation effectively divides the model into geometrically distinct parts, as illustrated in Figure 11(b) of the main text.

Next, we consider the other dependency graphs, $G_{C,I} = (V, E_C \cup E_I)$. Our goal is to merge nodes with similar dependency relationships, thereby minimizing the overall graph size without significantly affecting the final result.

Specifically, as we traverse the edges of G_H, we assess whether the dependency relationships between the dependency and dependent nodes are similar in $G_{C,I}$. For instance, if an edges (c_i, c_j) exists in G_H, and at the same time, both $(c_i, c_k) \in G_{C,I}$ and $(c_j, c_k) \in G_{C,I}$ hold, then (c_i, c_j) will not be marked. Similarly, if there exist edges $(c_i, c_{k_1}) \in G_{C,I}$ and $(c_i, c_{k_2}) \in G_{C,I}$, and an edge (c_{k_1}, c_{k_2}) exists in G_H, then (c_i, c_j) will also not be marked.



The above describes the case where both c_i and c_j act as dependency in $G_{C,I}$. The same logic applies when c_i and c_j is itself a dependent. Ultimately, all height dependency edges that do not satisfy these conditions are marked (see Figure 11(c) of the main text). After marking all the edges in G_H, we perform a DFS on G_H. During traversal, all notes encountered before reaching a marked edge are merged into a subgraph. Finally, each simplified subgraph is treated as a single node to construct a new graph G^* (see see Figure 11(d) of the main text), which will be used in the subsequent shortest travel sequence search algorithm.

Controllability of graph simplification. To allow manual control over the degree of graph simplification, we introduce a parameter $\Gamma \in [0, 1]$, where 0 correspond to G and 1 corresponds to current G^*, as shown in Figure 27. The key idea of controllable graph simplification is to use Γ to further subdivide already simplified subgraphs. Theoretically, the number of segments a subgraph should be divided

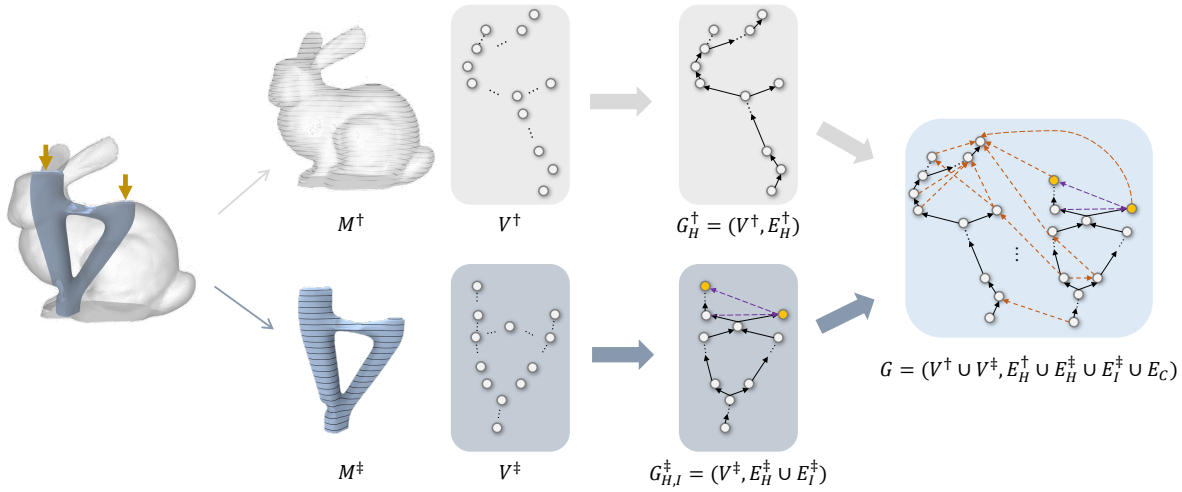


Figure 25: Generation of the dependency graph for multi-cavity models. First, the multi-cavity model is divided into two models, $M^\dagger$ and $M^\ddagger$, according to non-injection cavities and injection cavities. Slicing is carried out separately to generate the contours, and simultaneously, the contours serves as the nodes $V^\dagger$ and $V^\ddagger$ of graph. For model $M^\dagger$, only the height dependency graph $G_H^\dagger$ is generated. For model $M^\ddagger$, on the basis of generating the height dependency graph $G_H^\ddagger$, an injection dependency graph $G_I^\ddagger$ also needs to be generated. Finally, the two graphs $G_H^\dagger$ and $G_{H,I}^\ddagger$, are merged, and the collision-free dependency graph G_C is generated uniformly. Eventually, the final dependency graph G is produced.

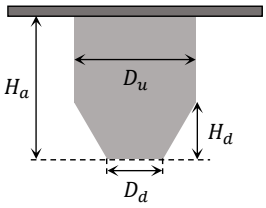


Figure 26: Print nozzle shape. Here, H_a denotes the total height, H_d is the lower half height (typically set to half of the total height), D_d is the nozzle diameter, and D_u is the upper diameter.

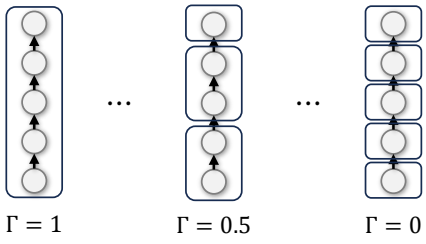


Figure 27: Controllability of graph clustering. As the value of Γ ranges from 1 to 0, the complexity of the graph G^* increases.

into is given by:

$$N_{split} = \lfloor (|V^*| - 1)(1 - \Gamma) + 1 \rceil, \tag{10}$$

where $|V^*|$ represents the number of nodes in a simplified subgraph. In practice, since subgraphs vary in size, the interval for splitting elements within a subgraph is determined as:

$$\Delta_{split} = \left\lceil \frac{|V^*|}{N_{split}} \right\rceil. \tag{11}$$

This ensures that larger subgraphs are divided into more segments, while smaller subgraphs remain relatively intact, allowing for flexible control over the final graph complexity.

E BEAM SEARCH

Initialization. We start with an empty sequence σ and identify all nodes in G^* with an in-degree of 0. These nodes form the initial candidate set. Instead of considering all possible expansions, we limit the number of explored branches using a predefined beam width $\mathcal{W}$, ensuring a manageable search space.

Expansion and pruning. At each iteration, we expand the current partial solutions by selecting a subset of the most promising candidate nodes based on an evaluation functions $f(\cdot)$. After selecting a candidate node, it is removed from G^*, potentially introducing new nodes with an in-degree of 0 into the candidate set. The beam width $\mathcal{W}$ determines how many candidates are retained at each step, allowing us to explore multiple paths while discarding suboptimal ones early. The scoring of each candidate node is computed as follows:

$$\min f(\sigma) = \mu \frac{f_1(\sigma)}{f_1(\sigma_0)} + (1 - \mu) \frac{f_2(\sigma)}{f_2(\sigma_0)}, \tag{12}$$

where:

$$f_1(\sigma) = \sum_{i=1}^{m-1} \|\text{pos}(c_{\sigma(i)}) - \text{pos}(c_{\sigma(i+1)})\| \tag{13}$$

measures the total idle travel distance, and

$$f_2(\sigma) = \sum_{i=1}^{m-1} \mathbb{I}(c_{\sigma(i)}, c_{\sigma(i+1)} \notin G_H) \tag{14}$$

counts the number of long-jump for 3D printing nozzle, where $\mathbb{I}(\cdot)$ is an indicator function defined as:

$$\mathbb{I}(X) = \begin{cases} 1, & \text{if } X \text{ is true,} \\ 0, & \text{otherwise.} \end{cases} \tag{15}$$

Here, σ_0 represents default sort of the traditional manufacturing, and μ is a weighting factor balancing the two objectives.

Termination condition. The process continues until all nodes are included in a complete sequence. Since beam search explores multiple paths, we obtain a set of candidate solutions. The final manufacturing sequence is chosen by evaluation functions $f(\cdot)$. An example of our beam search algorithm is shown in Figure 12 of the main text.

F INJECTION MATERIAL VOLUME DETERMINATION

For each injection step, the required liquid volume is estimated by summing the internal volumes of all relevant contours. The internal volume of each contour is calculated as:

$$\text{Vol}(c_i^\dagger) = \frac{h}{2} \left| \sum_{k=1}^{n} (x(p_k)y(p_{k+1}) - x(p_{k+1})y(p_k)) \right| + \gamma, \tag{16}$$

where $p_{n+1} = p_1$. Here, $c_i^\dagger = R_i - c_i$ represents the contour after removing the previously filled area. The function $x(\cdot)$ and $y(\cdot)$ denote the coordinates of a point, while $\mathbf{p} = \{p_1, p_2, \ldots, p_k\}$ represents the set of points forming the contour $c_i^\dagger$. γ is the error of injection volume between theory and reality.

Thus, the volume of liquid injected at each injection contour $\mathbf{c}^*$ is given by:

$$I(c_i^*) = \sum_{k=1}^{s} \text{Vol}(c_k^\dagger) - I(c_{i-1}^*), \tag{17}$$

where s represents the set of all contours located below the current injection contour c_i^*. Additionally, the positioning of the injection nozzle is determined by the center v^* of the MIC within the injection contour, as described in Equation 3 of the main text.

When generating the G-code file at the end, during each idle travel, we raise the print head to the maximum height of the already printed contours. This measure is taken to prevent collisions between the print head and the printed contours during the idle traverse. This approach ensures the smooth progress of the manufacturing process.